Chapter 1

Introduction

1.1 Motivation

Sparse matrix–dense matrix multiplication (SpMM) is one of the fundamental computational kernels in modern machine learning and graph-based computing. In graph neural networks (GNNs) it embodies the neighbourhood aggregation step, in which a sparse adjacency matrix is multiplied against a dense feature matrix [1, 2]. In large-scale recommender systems, collaborative filtering relies on sparse user–item interaction matrices whose dimensions can reach hundreds of millions of rows [3]. In scientific simulation it is the inner loop of iterative solvers for finite-element and finite-difference discretisations. The same pattern appears in transformer attention with sparse masks and in the weight-matrix multiply step of pruned and compressed neural networks [4]. Across all of these workloads the common denominator is that the useful arithmetic is only a small fraction of the work that a general-purpose processor actually performs, with most cycles spent chasing indirect indices, handling row boundaries, and issuing scattered memory accesses.

With the rapid rise in demand for AI, there has been a matching rise in demand for better model algorithms, more data, and above all more compute. On the compute side, most AI workloads are now executed on application specific integrated circuits (ASICs), since these have been shown to maximise throughput, energy efficiency, and cost effectiveness for the particular matrix operations that AI models need to run. Some of the most well known commercial examples are the Tensor Processing Unit (TPU) from Google and the Neural Processing Unit (NPU) from Apple. The design of any such ASIC, however, almost always begins with prototyping on

a Field Programmable Gate Array (FPGA), because FPGAs can execute many operations in parallel and are easily reconfigurable, which keeps the cost and turnaround time of architectural exploration both low and fast. An FPGA-hosted accelerator for SpMM is therefore a natural first step towards a fully custom silicon implementation, and is the platform targeted by this project.

1.2 Project Aim

The aim of this project is to design, implement, and evaluate a RISC-V compatible hardware accelerator for compressed sparse row (CSR) SpMM on the Terasic DE1-SoC FPGA board. The accelerator must integrate cleanly into a small system-on-chip (SoC), share on-chip block RAM with the CPU, and deliver a measurable, reproducible speedup over a comparable software implementation while remaining understandable at the register-transfer level (RTL) and transparent in its benchmarking methodology. A secondary aim is to isolate the individual contribution of each optimisation step, so that the dominant performance bottleneck can be identified through direct measurement rather than assumption. To achieve this, the design is developed in three stages (a baseline accelerator, a parallel DSP-oriented multiply-accumulate enhancement, and a symmetric INT8 packing stage), forming a controlled experimental path in which only one design dimension is changed at a time.

Beyond the core aim, the project defined three stretch goals at the outset. The first was to introduce a custom RISC-V instruction to trigger the accelerator directly from the CPU pipeline, rather than through a memory-mapped register write, which would reduce launch overhead and demonstrate that the PicoRV32 softcore can be extended at the instruction set level. The second was to implement runtime INT8 quantisation so that operand fetch bandwidth could be reduced by packing four quantised values into a single 32-bit RAM word. The third was to extend the accelerator with optional activation functions and a max-pooling stage, so that it could serve as a building block for a small inference pipeline rather than as a standalone kernel. Of the three, the INT8 quantisation goal was achieved and is reported in full in this work; the remaining two are discussed as future work in Chapter 8.

1.3 Report Structure

The remainder of this report is structured as follows. Chapter 2 reviews the background needed to justify the design choices, covering SpMM workloads, sparse data formats, prior accelerator work, the rationale for FPGAs, and INT8 quantisation theory. Chapter 3 defines the implemented SoC platform and its memory-mapped I/O interface. Chapter 4 presents the accelerator architecture and each optimisation step in detail. Chapter 5 defines the hardware verification approach and the unit and integration testbenches used to check the RTL. Chapter 6 describes the bare-metal firmware, the benchmark suite, and the measured cycle-count and correctness results across the three optimisation stages. Chapter 7 discusses lessons learned and project management, and Chapter 8 closes the report with a summary and proposed future work.

Chapter 2

Background and Literature Review

There is a vast body of algorithms for Sparse Matrix Multiplication (SpMM), as well as many different storage formats for converting sparse matrices into representations that can be operated on efficiently. This chapter provides the information needed to understand the context and design decisions taken throughout this project, alongside the goals that were set out at the start. If any of the material that follows is unfamiliar, a more elementary overview of SpMM and the CSR format is given in Appendix **??**, and further reading can be pursued through the references cited in each section.

2.1 SpMM in Modern Workloads

The motivation for a dedicated SpMM accelerator is made concrete by the sparsity levels that real workloads exhibit. Figure 2.1 summarises representative density ranges across four workload classes: in each case, the fraction of nonzero entries is small enough that storing and operating on the matrix in dense form would be wasteful by one to five orders of magnitude.

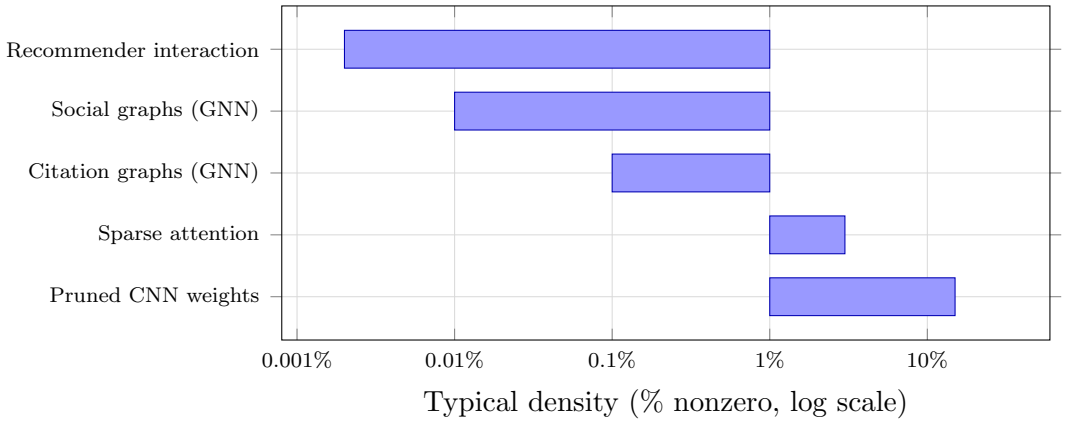

FIGURE 2.1: Representative density ranges for matrices encountered in four classes of modern workload. Social and citation graphs in GNN benchmarks such as `ogbn-arxiv` and `Reddit` typically have densities below 0.1% [1]; large-scale recommender interaction matrices are often two orders of magnitude sparser still [3]. Pruned CNNs sit at the other end of the range, retaining 10–20% of their weights after aggressive pruning [4]. In every case, an SpMM execution model that iterates only over the nonzeros avoids between 90% and $> 99.99\%$ of the arithmetic that a dense multiply would perform.

2.1.1 Graph Neural Networks

Graph neural networks (GNNs) are an important and growing class of workloads in which SpMM appears as a first-class primitive. Each layer of a message-passing GNN such as GraphSAGE [1] or a Graph Attention Network [2] computes

$$H^{(l+1)} = \sigma\left(\hat{A}\, H^{(l)}\, W^{(l)}\right),$$

where $\hat{A}$ is the normalised sparse adjacency matrix of the input graph and $H^{(l)}$ is the dense feature matrix. The product $\hat{A}\, H^{(l)}$ is a sparse–dense multiply that dominates inference time on large graphs.

2.1.2 Recommender Systems and Sparse Embedding Lookup

Industrial recommender systems such as the YouTube recommendation engine [3] maintain embedding tables with hundreds of millions of rows. The forward pass reduces to a row-gather SpMM on the interaction matrix, where sparse interaction patterns meet dense embedding rows.

2.1.3 Scientific Computing

In finite-element and finite-difference simulation, sparse matrices arise from the discretisation of differential operators on computational meshes. SpMM appears in multivector iterative solvers such as block GMRES and Lanczos methods, where each iteration applies the sparse system matrix to a block of dense vectors.

2.1.4 Compressed and Pruned Neural Networks

Weight pruning removes less-significant connections from a dense neural network, turning weight matrices into sparse ones. Han et al. [4] showed that networks pruned to 80–90% sparsity can be executed in compressed form with large energy and storage savings; combined with INT8 quantisation, as in the EIE design, SpMM on the compressed weights becomes the dominant kernel. The same principle applies to sparse-attention transformers.

2.2 Why Sparsity Changes the Design Problem

2.2.1 The FLOP-to-Bandwidth Mismatch

Dense matrix multiplication is efficient because it has high operational intensity: for an $M \times K \times N$ General Matrix Multiply (GEMM), many Floating Point Operations (FLOPs) are performed for every byte of data loaded from memory, with each operand reused across the N output columns. Sparse matrix multiplication breaks this reuse pattern. For a CSR SpMM with NNZ nonzeros, the number of useful MACs per byte of CSR index data fetched is

$$\mathrm{OI} = \frac{2 \cdot \mathrm{NNZ} \cdot N}{\mathrm{NNZ} \cdot (4+4) + (M+1) \cdot 4 + \mathrm{NNZ} \cdot 4 \cdot N},$$

where the denominator accounts for `colidx` (4 bytes per entry), `values` (4 bytes per entry), `rowptr` (4 bytes per pointer), and the dense B segment reads (4 bytes per column per nonzero). For small N and low density, the operational intensity is significantly below the roofline knee of typical on-chip BRAM systems [5], indicating that SpMM is memory-bandwidth-limited rather than compute-limited on most platforms.

2.2.2 Irregular Access and Control-Flow Overhead

Beyond raw bandwidth, SpMM imposes control overhead that dense accelerators are not designed to handle. For each nonzero a_{ij}, the hardware must:

1. read the column index j from `colidx`;
2. compute the address of the relevant row of B as B_base $+ j \cdot N$;
3. read the N values of $B[j, :]$; and
4. accumulate $a_{ij} \cdot B[j, n]$ into $C[i, n]$ for each column n.

Steps (1) and (2) introduce indirection, often called "pointer-chasing": the next address depends on a value that has just been read. This breaks prefetching, and on an in-order core such as PicoRV32 each stall is exposed directly in the cycle count, so control overhead rather than arithmetic becomes the primary bottleneck. Figure 2.2 contrasts the predictable stride of a dense traversal with the indirect, data-dependent address sequence that CSR SpMM produces.

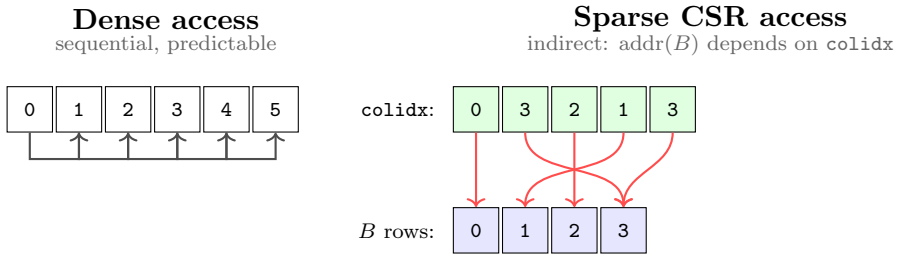

FIGURE 2.2: Memory access patterns in dense versus sparse CSR traversal. Dense access strides sequentially through contiguous addresses, so a prefetcher can fetch ahead. CSR access loads a column index from `colidx` first, then uses that value as an indirect pointer into B; the next address cannot be known until the previous load completes, which breaks prefetching and produces data-dependent cache misses.

2.3 Sparse Matrix Storage Formats

Several sparse matrix storage formats have been proposed; the choice of format strongly affects the hardware control structure. Table 2.1 summarises the most relevant formats and their suitability for hardware acceleration.

Format	Acronym	Description	Hardware suitability
Coordinate list	COO	Stores (row, col, val) triplets unsorted or sorted by row. Simple to generate; requires sorting for efficient row-wise access.	High overhead per nonzero; poor for row-streaming hardware.
Compressed Sparse Row	CSR	Three arrays: `rowptr` (M+1 entries), `colidx` (NNZ), `values` (NNZ). Row bounds are contiguous.	Excellent for row-wise traversal; natural hardware FSM mapping.
ELLPACK	ELL	Pads each row to the maximum NNZ per row; stores in column-major order.	Good for regular sparsity; wastes storage and bandwidth when rows vary widely.
Block Compressed Sparse Row	BSR	Groups nonzeros into dense $r \times c$ blocks stored in CSR form.	High throughput on block-sparse matrices; unsuitable for unstructured sparsity.
Compressed Sparse Column	CSC	Transpose of CSR; columns are compressed.	Natural for column-wise access patterns; awkward for row-oriented output.

TABLE 2.1: Comparison of common sparse matrix storage formats and their suitability for hardware acceleration.

2.3.1 Why CSR Was Chosen

Figure 2.3 shows the CSR encoding of a small 4×4 sparse matrix. The three arrays `rowptr`, `colidx`, and `values` together record exactly which nonzeros exist and where, with no storage spent on the zero entries.

CSR was selected for this project because it maps directly onto the natural execution order of row-wise SpMM. Gustavson's algorithm for sparse matrix multiplication [6] established that row-wise traversal of CSR is asymptotically optimal for output-row-oriented computation, and that the row pointer array provides $O(1)$ access to the start of each row's nonzero sequence. Bell and Garland [7] demonstrated that CSR remains the most widely supported format for both CPU and GPU SpMV/SpMM implementations precisely because its memory layout matches the natural iteration over output rows.

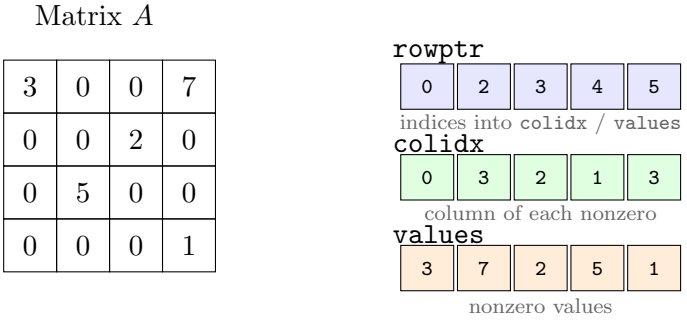

FIGURE 2.3: CSR encoding of a 4×4 sparse matrix with five nonzeros. `rowptr`[i] is the index of the first nonzero in row i; row i owns entries `rowptr`[i] through `rowptr`[i+1]−1 in both `colidx` and `values`. `rowptr`[M]=NNZ by convention.

For a hardware FSM, the CSR structure offers a critical advantage: the bounds of each row's nonzero segment are available with a single pointer read, and the entire nonzero segment can be streamed sequentially from memory. There is no need for global index sorting, block alignment padding, or two-dimensional address translation. The firmware preparation cost is also low: generating CSR from a list of nonzeros requires a single pass to count per-row nonzeros and a second pass to write values and indices.

2.4 SpMM Execution with CSR

The previous section explained how CSR encodes a sparse matrix. This section explains how a sparse–dense matrix multiply actually uses that encoding. The distinction matters for hardware, because the same three arrays that describe the storage of A also drive the address generation of the accelerator.

2.4.1 Problem Statement

Given a sparse matrix $A \in \mathbb{R}^{M \times K}$, a dense matrix $B \in \mathbb{R}^{K \times N}$, and an output $C \in \mathbb{R}^{M \times N}$, the product $C = AB$ is defined element-wise by

$$C[i, n] \;=\; \sum_{j=0}^{K-1} A[i, j] \cdot B[j, n].$$

For a dense A the inner sum performs K multiplies per output element, the vast majority of which are zero when A is sparse. CSR removes those zeros from the

loop entirely: only the nonzero entries of each row of A are enumerated, and each one contributes a scaled copy of a single row of B to the corresponding row of C.

2.4.2 A Concrete Example

Figure 2.4 shows a 4×4 sparse A matrix, multiplied by a 4×3 dense B to produce a 4×3 output C. A has five nonzeros, so only five sparse–dense interactions occur in the entire multiply. Each one takes a single nonzero of A and a single row of B (the row whose index equals the column index of the nonzero) and produces a scaled row that is accumulated into the matching row of C. The highlighted entry $A[0,3] = 7$ picks row 3 of B, multiplies it by 7, and contributes $(70, 77, 84)$ to row 0 of C; $A[0,0] = 3$ contributes a further $(3, 6, 9)$ from row 0 of B, giving the final $C[0,:] = (73, 83, 93)$.

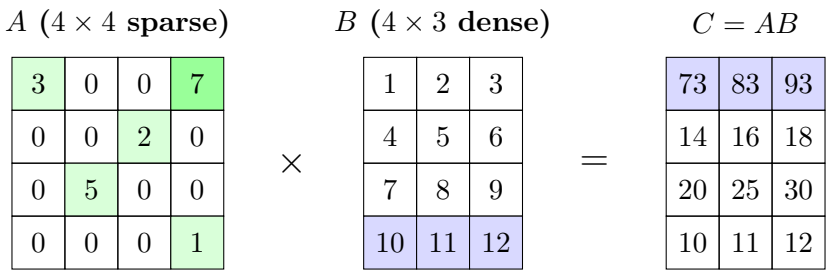

FIGURE 2.4: A small sparse–dense matrix multiply $C = AB$. Shaded cells of A are its five nonzero entries; the darker cell $A[0,3] = 7$ is highlighted as a worked example. Each nonzero of A scales the row of B whose index equals its column and accumulates that scaled row into the matching row of C.

2.4.3 Row-Wise CSR Execution

In CSR form, the nonzeros of A are stored in row-major order as described in Section 2.3. The computation of $C = AB$ iterates over each output row and then over each nonzero in that row:

1. For each output row $i = 0, 1, \ldots, M-1$, look up the row bounds $p_{\text{start}} =$ `rowptr`$[i]$ and $p_{\text{end}} =$ `rowptr`$[i+1]$.
2. For each $p \in [p_{\text{start}}, p_{\text{end}})$, read the column $j =$ `colidx`$[p]$ and the scalar $a =$ `values`$[p]$, fetch the dense row $B[j,:]$, and accumulate $C[i,:] \mathrel{+}= a \cdot B[j,:]$.

The inner update is a scalar-times-vector operation across the N output columns, so one nonzero of A produces N useful multiplies in parallel, and the sparse dimension K never appears as a loop variable. The parallelism in the hardware comes from the dense dimension, where the work is regular, rather than the sparse dimension, where it is not.

Table 2.2 traces the two iterations that compute row $i = 0$ of the example. The bounds `rowptr`$[0] = 0$ and `rowptr`$[1] = 2$ select two nonzeros from `colidx` and `values`. The first iteration uses `values`$[0] = 3$ and `colidx`$[0] = 0$ to fetch $B[0, :]$ and accumulate $3 \cdot B[0, :]$ into $C[0, :]$. The second iteration uses `values`$[1] = 7$ and `colidx`$[1] = 3$ to fetch $B[3, :]$ and accumulate $7 \cdot B[3, :]$. The two partial updates sum to $C[0, :] = (73, 83, 93)$, agreeing with Figure 2.4.

p	`values`[p]	`colidx`[p]	$B[j, :]$	$a \times B[j, :]$	$C[0, :]$ after update
0	3	0	$(1, 2, 3)$	$(3, 6, 9)$	$(3, 6, 9)$
1	7	3	$(10, 11, 12)$	$(70, 77, 84)$	$(73, 83, 93)$

TABLE 2.2: Walk-through of the row-wise CSR execution for output row $i = 0$ of Figure 2.4. Each iteration reads one nonzero of A through the CSR arrays, fetches the corresponding row of B, and accumulates the scalar-scaled vector into $C[0, :]$.

2.4.4 Why This Maps Well to Hardware

The execution model has three properties that suit a small FPGA accelerator:

1. *Address generation from three arrays.* `rowptr` supplies the loop bounds for each output row, `colidx` supplies the row index of B to fetch, and `values` supplies the scalar multiplier. No global index sort, block alignment, or two-dimensional address translation is required.

2. *Data-parallel inner update.* The scalar-vector update maps onto an array of TN parallel MAC units, each handling one column of the output tile, so the MAC array sees regular work even when the outer traversal is irregular.

3. *Amortised irregular cost.* Each operand fetched from `colidx` and `values` drives N multiplies in the output, so the cost of the indirect CSR lookup is spread over all N output columns.

2.5 Prior Work on SpMM Accelerators

2.5.1 OuterSPACE

OuterSPACE [8] targets the SpGEMM (sparse–sparse matrix multiply) problem using an outer-product decomposition. Rather than traversing CSR rows serially, it computes partial products for each nonzero column of A and accumulates them into a merge tree, achieving high throughput by parallelising across columns of A and using dedicated on-chip accumulators.

2.5.2 SpArch

SpArch [9] also targets sparse–sparse computation, focusing on the merger phase that reduces partial products to a final output. It introduces a condenser that reorganises partial products to maximise row-merger bandwidth, and shows that the merger step, not the multiplication step, typically dominates sparse–sparse computation time.

2.5.3 ExTensor

ExTensor [10] generalises sparse computation to higher-order tensors and introduces intersection hardware that skips pairs of zero operands before they reach the multiplier array, avoiding the memory read entirely when a nonzero pair is absent. CSR provides a weaker form of this by construction: the column-index array only points to nonzero entries, and zero rows of A produce no traversal steps at all.

2.5.4 SIGMA

SIGMA [11] targets the sparse-weight GEMM that arises during DNN training after unstructured pruning. It uses a flexible *Forwarding Adder Network* (FAN) as a reduction tree whose connectivity is reconfigured at runtime to match the nonzero pattern of the weight matrix, showing that hardware-reconfigurable reduction topologies can absorb the irregular sparsity of training-time gradients.

2.5.5 Positioning This Work

Taken together, the four designs span complementary regions of the sparse-acceleration space. OuterSPACE and SpArch prioritise high-throughput sparse–sparse accumulation, with SpArch specifically attacking the merger phase. ExTensor emphasises operand intersection and zero-pair skipping. SIGMA targets reconfigurable reduction for sparse DNN training. Each assumes a throughput regime, scheduling mechanism, or memory hierarchy that is not available on a Cyclone V FPGA. This project instead targets the simpler but practically relevant CSR SpMM case on a resource-constrained FPGA SoC, aligning the storage format, datapath, and quantisation strategy with the dominant cost of the workload.

2.6 Hardware Platform Alternatives

General-purpose CPUs handle SpMM inefficiently because the irregular access pattern defeats prefetching and out-of-order speculation, and on a small in-order soft-core such as PicoRV32 there is no data cache at all. GPUs achieve high SpMM throughput through massive parallelism [7] but require 100–400 W and a substantial runtime stack, which rules them out for an embedded SoC. A custom ASIC would deliver the best throughput and power, but the development cost and fabrication lead time are impractical for a final-year project.

FPGAs provide a practical middle ground: custom datapath parallelism, flexible on-chip memory organisation, and far lower power than a GPU [12]. The Cyclone V on the DE1-SoC supplies M10K BRAM blocks, DSP multiplier-accumulator blocks, and configurable logic that can be combined into a bespoke CSR SpMM datapath with a memory interface tailored to the access pattern, which is a practical choice for prototyping an accelerator architecture before any ASIC commitment.

2.7 INT8 Quantisation

2.7.1 Motivation

Quantisation reduces the numerical precision of stored values, decreasing memory footprint and potentially increasing arithmetic throughput [13, 14]. For SpMM specifically, quantisation acts directly on the bottleneck: an un-packed 32-bit word

carries one operand, whereas INT8 packing places four operands in the same word. For a memory-bound kernel this reduces the number of RAM reads needed to fill an operand tile by the same factor, which is the dominant reason INT8 is attractive for SpMM. The concrete cycle-level implications on the accelerator datapath are developed in Chapter 4.

2.7.2 Symmetric Linear Quantisation

The project adopts runtime symmetric linear quantisation. Given a vector $\mathbf{v}$ of full-precision values, the maximum absolute value $v_{\max} = \max_i |v_i|$ is computed, and each value is mapped to a signed 8-bit integer by

$$q_i = \text{clip}\left(\text{round}\left(\frac{v_i}{v_{\max}} \cdot 127\right), -127, 127\right), \tag{2.1}$$

where the scale factor is $s = v_{\max}/127$. Reconstruction uses $\hat{v}_i = q_i \cdot s$, so the quantisation error is bounded by $|e_i| \leq s/2$. Symmetric quantisation was preferred over asymmetric because it eliminates the need for a zero-point offset in the inner MAC loop, simplifying the hardware accumulation logic.

2.7.3 Accumulation Precision

INT8 accumulation into INT8 is insufficient for most practical workloads: a sum of K products of INT8 values each up to ± 127 can reach $\pm 127^2 \cdot K = \pm 16{,}129K$, which overflows INT8 and INT16 even for moderate K. The present design therefore accumulates into INT32, which can represent sums of up to $2^{31}/16{,}129 \approx 133{,}000$ elements before overflow. This matches the widely adopted practice of INT8×INT8 with INT32 accumulation as described by Jacob et al. [13].

2.8 Roofline Analysis and Design Implications

The roofline model [5] characterises whether a kernel is compute-bound (throughput limited by the peak arithmetic rate) or memory-bound (throughput limited by available bandwidth). The position of a kernel on the roofline is determined by its *arithmetic intensity*: the ratio of arithmetic operations to bytes of data movement. A kernel whose intensity places it to the left of the roofline knee is memory-bound,

and its achievable throughput scales linearly with bandwidth; one placed to the right is compute-bound, and its throughput is capped by the peak arithmetic rate.

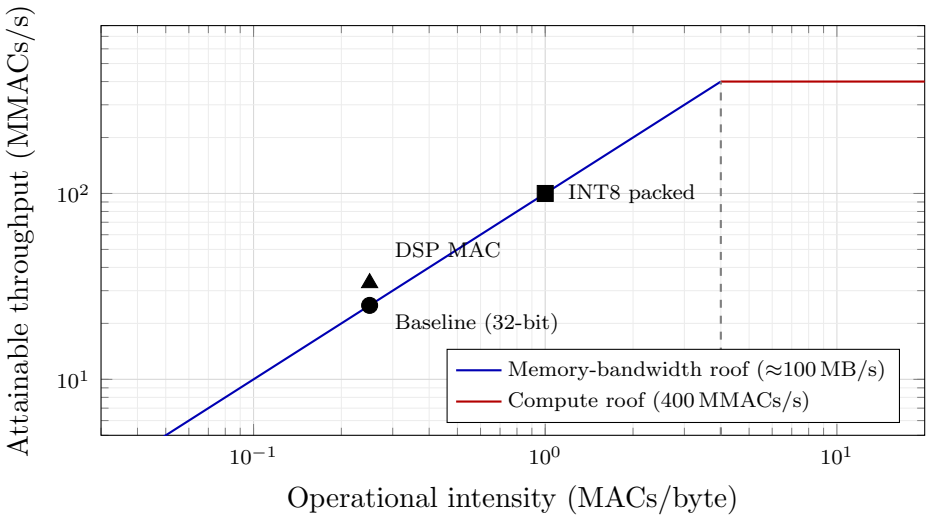

FIGURE 2.5: Roofline interpretation of the three accelerator stages on the Cyclone V platform. The baseline and DSP-enhanced designs sit on the memory-bandwidth roof: arithmetic throughput is capped by operand fetch, so widening the MAC array yields only a small gain. INT8 packing raises the operational intensity by a factor of four (four operands per 32-bit word), shifting the design rightward and upward along the bandwidth roof towards the compute-bound region.

SpMM sits firmly in the memory-bound regime for the reasons developed in Section 2.2: low operand reuse and indirect access keep the arithmetic intensity well below the knee. The practical implication for accelerator design is that widening the MAC array yields only a small gain unless operand supply is widened alongside it. Quantisation addresses this directly by increasing the number of useful operands packed into each memory transaction, moving the design's operating point along the bandwidth roof towards the compute-bound plateau. A numerical instantiation of this argument for the present $TN = 8$ accelerator is given in Chapter 4, and Figure 2.5 illustrates the qualitative shift that INT8 packing produces.

Chapter 3

System-on-Chip Design

This chapter describes the system that hosts the SpMM accelerator: what the SoC looks like as a whole, why a soft RISC-V core was chosen as the host, how memory and the bus are organised, and how firmware drives the accelerator through a small bank of memory-mapped registers. The accelerator itself is treated as a black box here; its internal datapath and optimisation are the subject of Chapter 4.

3.1 System Overview

The SoC is synthesised entirely into FPGA fabric on the Terasic DE1-SoC development board [15]. The hard ARM Cortex-A9 on the HPS side of the device is unused, so the whole design is self-contained in programmable logic. All benchrmark code and data reside on chip: firmware, CSR arrays, the dense matrix B, and the output matrix C all reside in a shared 64 KB dual-port block RAM. The entire SoC is clocked from the DE1-SoC's 50 MHz on-board oscillator.

Keeping the design inside the FPGA has three consequences that shape the rest of this chapter. First, there is no AXI bridge, no Linux boot, and no HPS memory management to reason about, so cycle counts taken on the PicoRV32 core are directly comparable with cycle counts taken on the accelerator. Second, the problem sizes are bounded by on-chip capacity rather than by external DRAM bandwidth, which removes one variable from the comparison at the cost of limiting absolute problem size. Third, the CPU and accelerator share the same on-chip memory, which makes the dual-port arrangement described in Section 3.4 the central architectural decision of the platform.

Figure 3.1 shows the top-level block diagram. The PicoRV32 CPU, the SpMM accelerator, UART, and GPIO all hang off a simple interconnect that routes requests to the BRAM or to one of the memory-mapped peripheral regions.

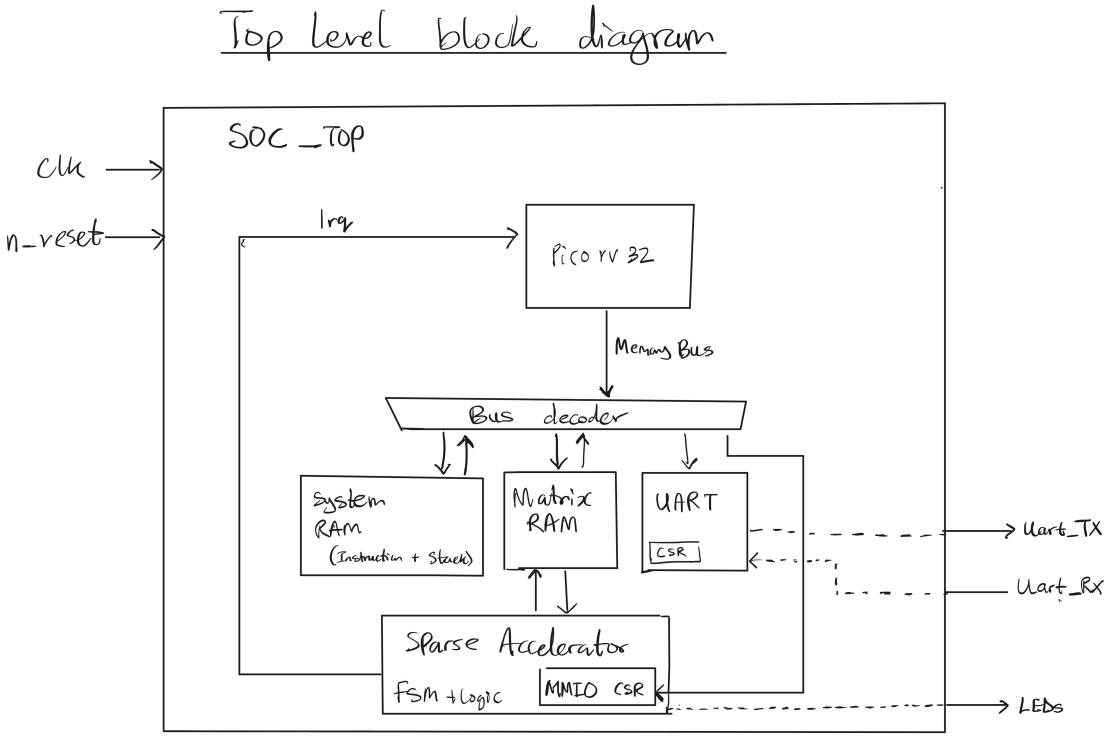

FIGURE 3.1: Top-level RISC-V SoC block diagram.

3.2 Why a Soft RISC-V SoC

The host processor is PicoRV32 [16], a compact RV32IM soft-core implementing the RISC-V base integer instruction set with the hardware integer multiply extension [17]. Three practical reasons drove the choice.

PicoRV32 is small, roughly 700 to 1000 ALMs depending on configuration, which leaves the majority of the Cyclone V fabric available for the accelerator and the 64 KB BRAM. It exposes the `rdcycle` performance counter CSR natively, so all benchmark timing in this report is cycle-accurate without needing an external timer or off-chip instrumentation. And its memory bus is a plain valid/ready handshake with address, data, and write-enable signals, which drops straight onto the SoC interconnect without an AXI adapter.

The important caveat for interpreting the results later in this report is that PicoRV32 is in-order and non-pipelined. It issues one instruction at a time and stalls fully on every load. For a synchronous BRAM access, the CPU waits one cycle after presenting the address before the data is returned. This makes the software baseline slower than a modern superscalar CPU would be, which means the reported accelerator speedup is larger than it would be against, say, a Cortex-A class core. The comparison is still meaningful because the goal is to quantify the benefit of custom SpMM hardware against the same RV32IM ISA, not to claim absolute superiority over a desktop processor.

The M extension matters for the same reason. The software SpMM reference uses hardware `MUL` instructions for the inner multiply-accumulate. Without it, PicoRV32 would fall back to a software multiply routine that is typically ten to thirty times slower, which would artificially inflate every speedup number in Chapter 4. The M extension is therefore a prerequisite for a credible software baseline.

3.3 Target FPGA Platform

The Cyclone V 5CSEMA5F31C6 on the DE1-SoC provides the resources summarised in Table 3.1 [18]. The estimated utilisation column reports the design's approximate footprint; final synthesis numbers are pending completion of the Quartus timing closure study.

Resource	Total available	Used (estimated)
Adaptive Logic Modules (ALMs)	32,070	≈5,000 (16%)
Logic registers	128,280	≈8,000 (6%)
M10K memory blocks (10 Kbit each)	397	≈20 (5%)
DSP 18×18 multiplier blocks	87	≈8 (9%)
I/O pins	457	—

Table 3.1: Cyclone V 5CSEMA5F31C6 resource availability and estimated design utilisation.

Two resource classes are particularly important for this design. The M10K blocks each provide 10 Kbits (roughly 1.25 KB) of configurable on-chip memory [19]. In true-dual-port mode with 32-bit-wide ports, each M10K holds 256 words, so the 64 KB shared BRAM needs around 16 M10Ks arranged as a cascade. That leaves plenty of headroom for any additional buffers the accelerator might want to add later. The DSP blocks provide 18×18-bit signed multipliers with a dedicated

accumulator register [20], which maps cleanly onto INT32 MAC operations and, with the packed-byte treatment described in Chapter 4, onto INT8 MACs as well.

3.4 Memory and Interconnect Architecture

3.4.1 Bus Protocol

The SoC interconnect uses the native PicoRV32 memory bus: a valid/ready handshake with a 32-bit address bus, a 32-bit bidirectional data bus, a 4-bit byte-enable signal, and a read/write indicator. Everything is synchronous to the 50 MHz system clock. Access latency is one cycle for BRAM (using the BRAM's registered output) and combinatorial for MMIO registers.

3.4.2 Dual-Port BRAM Organisation

The 64 KB on-chip RAM is implemented as an Altera `altsyncram` true-dual-port memory with 32-bit-wide ports. Port A belongs to the CPU and supports byte-granular writes through a 4-bit byte-enable mask. Port B belongs to the accelerator and uses 32-bit word-granular access. Figure 3.2 shows the arrangement.

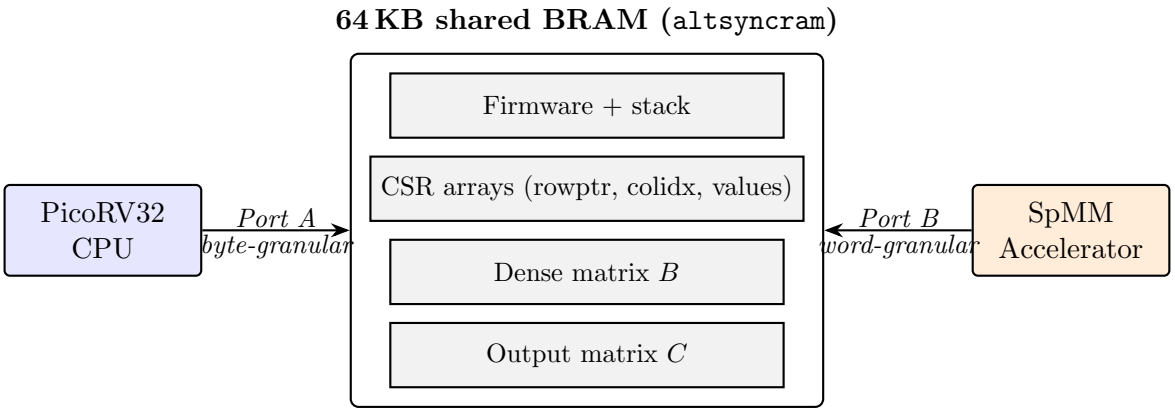

FIGURE 3.2: Dual-port BRAM organisation. The CPU and accelerator drive independent ports on the same memory array, so they can read or write different addresses on the same cycle without arbitration.

Because Port A and Port B address the same underlying array, two accesses that hit the same word on the same cycle produce undefined results. The firmware never writes to a buffer that the accelerator is actively reading, and it never reads C until the accelerator has signalled completion. Apart from that constraint, the two

sides are free to operate concurrently: during a benchmark run, for example, the CPU can stream the previous output over UART on Port A while the accelerator populates the next output on Port B.

3.4.3 Address Space

Table 3.2 summarises the full memory map, and Figure 3.3 shows the same information as a vertical strip for quick reference.

Address range	Peripheral	Purpose
`0x0000_0000--0x0000_FFFF`	On-chip RAM (64 KB)	Firmware code, stack, CSR arrays (`rowptr`, `colidx`, `values`), dense matrix B, output matrix C
`0x1000_0000--0x1000_00FF`	SpMM accelerator MMIO	Configuration registers, control, and status
`0x2000_0000--0x2000_000F`	GPIO	LED output, switch and button input, seven-segment display
`0x2000_0100--0x2000_01FF`	UART (`simpleuart`)	Serial output for UART benchmark reporting at 115200 baud

TABLE 3.2: System memory map of the implemented RISC-V SoC.

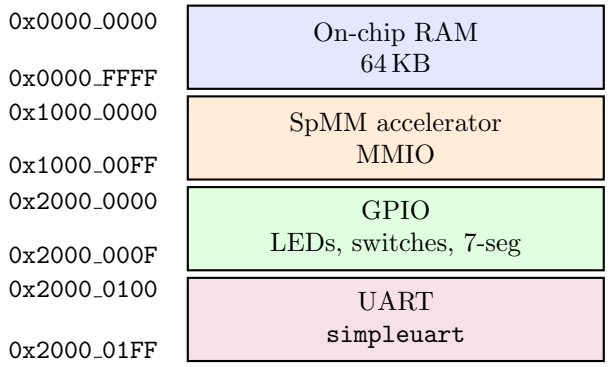

FIGURE 3.3: System memory map as a vertical address strip. Each region inside the interconnect is a simple comparison on the top byte of the address.

3.5 Accelerator Programming Interface

The accelerator is controlled entirely through memory-mapped registers at `0x1000_0000`–`0x1000_002C`. Firmware writes dimensions and base addresses, pulses the start bit, polls the status register, and reads the output once the done flag rises. There is no

DMA engine, no interrupt controller, and no driver layer; the whole interface is suitable for bare-metal firmware. Table 3.3 lists the register set.

Offset	Name	Description
0x00	CTRL	Control register: bit 0 = start, bit 1 = clear_done, bit 2 = irq_en, bit 3 = relu_en, bit 4 = int8_mode
0x04	STATUS	Status register: bit 0 = busy, bit 1 = done
0x08	M	Number of rows in A and C
0x0C	N	Number of columns in B and C (dense width)
0x10	K	Number of columns in A / rows in B
0x14	A_VAL_BASE	Base address of CSR values array
0x18	A_ROW_BASE	Base address of CSR rowptr array
0x1C	A_COL_BASE	Base address of CSR colidx array
0x20	B_BASE	Base address of dense matrix B (row-major)
0x24	C_BASE	Base address of output matrix C (row-major)
0x28	NNZ	Total number of nonzeros in A

TABLE 3.3: SpMM accelerator MMIO register map. All registers are 32-bit word-aligned. Base addresses are byte addresses into the shared 64 KB RAM.

3.6 System Operation

Figure 3.4 shows the handshake between firmware and accelerator for a single SpMM run. The CPU stages all inputs in BRAM, writes the MMIO configuration, pulses start, and then polls the status register. The accelerator runs asynchronously on Port B of the BRAM until it has written the last element of C, at which point it raises done. The CPU clears the flag and proceeds to read C.

In more detail, the accelerator is specified to:

1. read the matrix dimensions M, N, K, and NNZ from the MMIO registers when start is pulsed;
2. interpret A in CSR form using the three base-address registers (A_VAL_BASE, A_ROW_BASE, A_COL_BASE);
3. read the dense matrix B from shared RAM in row-major layout;
4. compute $C = A \times B$ using tiled parallel MAC operations across the dense-output dimension;
5. optionally apply element-wise ReLU to each output element before writeback when relu_en is asserted; and

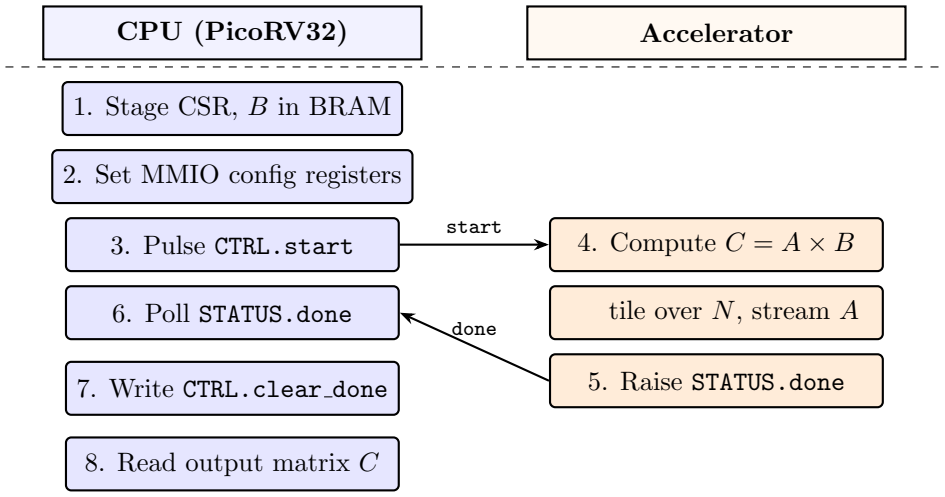

FIGURE 3.4: System operation sequence for a single SpMM run. Steps 1–4 and 7–8 execute on the CPU; steps 5–6 execute on the accelerator concurrently with the CPU's polling loop. The two lanes synchronise only through the `start` pulse and the `done` flag.

6. signal completion by asserting `STATUS.done` and releasing `STATUS.busy`.

When `CTRL.int8_mode` is set, the accelerator reinterprets the `values` array and matrix B as packed signed bytes, sign-extending each byte to 32 bits before multiplication. The packed-byte layout and its effect on BRAM bandwidth is analysed in Chapter 4.

3.7 Design Constraints and Implications

Four constraints shaped the platform decisions described above.

No external memory interface. All benchmark data lives in on-chip BRAM. This removes the SDRAM or DDR controller and the associated latency from the comparison, at the cost of bounding the problem size. In practice the firmware can size problems up to roughly $M = K = 64$, $N = 32$, depending on NNZ, before the combined CSR, B, and C footprint exceeds the 64 KB budget.

Simple MMIO control. The accelerator must be usable from bare-metal firmware with no OS, no DMA engine, and no driver stack. Register writes and a polling loop are enough, which is why the interface is intentionally small.

Tile-width constraint. The available DSP, BRAM, and routing budget on the Cyclone V limits how wide the accelerator can be made while still meeting timing

at 50 MHz. In the implemented design this leads to a tile width of $T_N = 8$, which provides useful parallelism across the dense-output dimension without overextending the platform resources. Chapter 4 justifies this choice in detail.

Reproducible cycle-accurate benchmarking. The firmware benchmark suite is deterministic (fixed PRNG seeds), self-verifying (element-by-element comparison against a software reference), and timed using PicoRV32's `rdcycle` CSR. Generating data at runtime rather than loading it from external storage was a direct consequence of the on-chip-only constraint.

Together these constraints explain the deliberate simplicity of the polling completion model, the modest tile width, and the on-chip data layout that the remainder of the report relies on.

Chapter 4

Accelerator Design

This chapter describes how the SpMM accelerator is built and why. It starts with a short overview of the architecture, works through the baseline datapath and the control FSM, and then covers the two design choices that do most of the work: the tile width and the row-wise CSR traversal. The final sections describe the DSP and INT8 optimisations and the physical-design constraints they exposed.

4.1 Overview

The accelerator has three jobs: walk the CSR structure of matrix A, stream row segments of matrix B into a local buffer, and accumulate products into a tile of the output matrix C. Everything else in the design follows from that pattern.

Rather than build one monolithic pipeline, the accelerator splits into two co-operating modules:

- `accel_ctrl`: a finite state machine that walks the CSR indices, issues RAM requests, and sequences the datapath.
- `accel_datapath`: the MAC array and the two local buffers (`B_Seg` and `C_Tile`) that hold the working set of a single tile.

A top-level wrapper (`accel_top`) connects the two blocks through a small MMIO front-end that decodes the PicoRV32 bus and surfaces the configuration registers listed in Table 3.3. The accelerator reaches memory through Port B of the

dual-port BRAM, which returns one 32-bit word per cycle with one cycle of registered read latency. Because that latency is fixed, every RAM access is scheduled deterministically and no stall/ready handshake is needed.

The remaining sections look at each piece in turn, starting with the datapath.

4.2 Baseline Datapath

The datapath is where the arithmetic happens. It is parameterised by three numbers: the tile width $TN = 8$ (the number of output columns processed in parallel), a maximum row count $M_{\max} = 64$, and a 32-bit signed operand width. Figure 4.1 shows the block diagram.

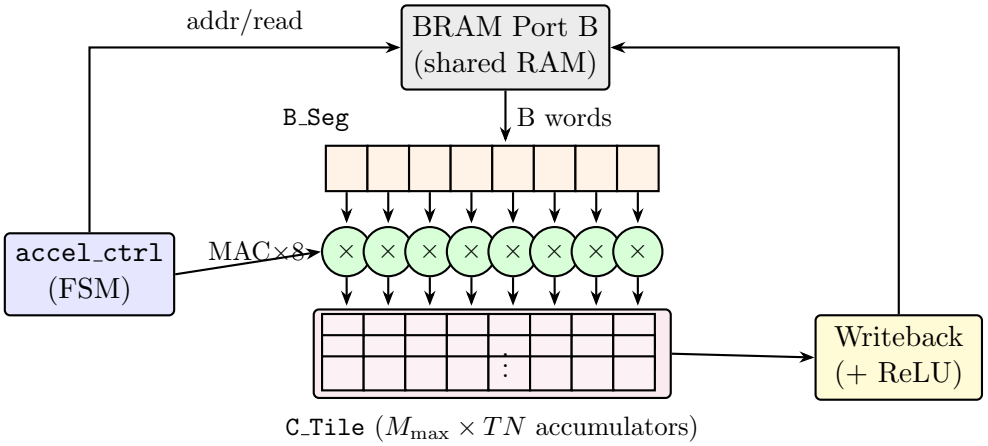

FIGURE 4.1: Baseline datapath. The controller fills `B_Seg` from Port B of the shared BRAM, drives the common operand `mac_a` to all eight MAC lanes, and selects the target row of `C_Tile`. Once a tile column is complete, the writeback path streams `C_Tile` back to memory, applying ReLU if enabled.

Two local buffers carry the state of a tile.

`B_Seg` is an eight-element array of 32-bit signed values that holds the row segment of B currently being consumed. For each nonzero a_{ij} in row i of A, the controller fills `B_Seg` with $B[j, c_0 : c_0{+}TN]$ before the MAC fires.

`C_Tile` is an $M_{\max} \times TN$ array of 32-bit signed accumulators that holds the partial sums for every row of C within the current tile column $[c_0, c_0{+}TN]$. It is implemented as fabric registers (around 256 ALMs on Cyclone V), not BRAM, so every entry can be read and updated on the same cycle. Keeping it flat, rather than as a set of per-row registers, matters because nonzeros from different rows

often land in the same tile column and the flat layout lets those updates proceed back-to-back with no writeback/restore cycles between them.

The core operation is one line. For each nonzero $a = A[i, k]$ the controller asserts `mac_en`, drives `mac_row` $= i$ and `mac_a` $= a$, and the datapath issues eight parallel MACs in a single clock:

$$\texttt{C\_Tile}[i][c] \;\mathrel{+}=\; a \times \texttt{B\_Seg}[c], \qquad c = 0, 1, \ldots, TN-1.$$

That expression shows where the parallelism comes from: a single nonzero in A meets an entire row segment of B, so one issue cycle produces TN output updates. The parallelism lies along the dense-output dimension, which is regular, rather than along the sparse dimension, which is not. Figure 4.2 shows the tile column inside C that this corresponds to.

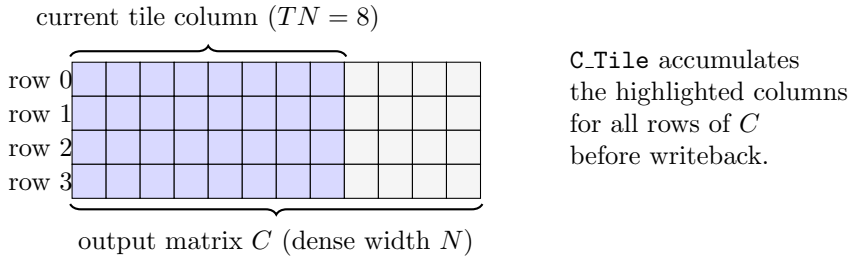

FIGURE 4.2: Tile column view. While the accelerator works on one tile column, the highlighted eight columns of C are held in `C_Tile` across all rows. Nonzeros from any row of A update their respective `C_Tile` row directly.

Once every row has been visited for the current tile column, the datapath walks `C_Tile` row by row and writes each entry back to shared RAM. If `CTRL.relu_en` is set, an element-wise $\max(0, x)$ is applied on the writeback path, so the accelerator can serve directly as the fully-connected layer of a neural-network inference pipeline.

4.3 Control FSM

The control FSM drives the datapath. It generates every RAM address, tracks the CSR traversal, and decides when to fill `B_Seg`, fire a MAC, or flush `C_Tile` back to memory. Table 4.1 lists its states, and Figure 4.3 shows the transitions.

In plain terms, the outer loop walks tile columns, the middle loop walks rows, and the inner loop walks the nonzeros of each row. For every nonzero the FSM fetches the column index and the value, fills `B_Seg`, fires one MAC cycle, and moves

State	Action
`IDLE`	Wait for `start` pulse. Latch M, N, K, NNZ, and base addresses from the MMIO register snapshot.
`INIT_TILE_CLEAR`	Clear all entries of `C_Tile` to zero in preparation for the first tile column ($M_{\max}$ cycles, one row per cycle).
`ROW_LOAD`	Read the CSR row bounds `rowptr[i]` and `rowptr[i+1]` for the current row i. Sub-phases `ROW_PHASE_0/1/2` absorb the registered RAM latency.
`NZ_FETCH`	For each nonzero in `[rowptr[i], rowptr[i+1])`: read `values[p]` and `colidx[p]` and compute the starting address of $B[\texttt{col}, :]$ for the current tile. Sub-phases `NZ_PHASE_0`–`NZ_PHASE_4` handle RAM latency and address arithmetic.
`MAC`	Fill `B_Seg` with TN consecutive values from B (one per cycle in INT32 mode, two reads total in INT8 mode), then assert `mac_en` for one cycle to update `C_Tile`.
`NEXT_ROW`	Increment the row index. If rows remain in this tile column, loop back to `ROW_LOAD`; otherwise proceed to `WRITE_TILE`.
`WRITE_TILE`	Write all $M \times TN$ entries of `C_Tile` back to shared RAM (with optional ReLU). Advance the tile-column offset by TN; if more tile columns remain, clear `C_Tile` and loop to `ROW_LOAD`, otherwise proceed to `DONE`.
`DONE`	Assert `STATUS.done` and release `STATUS.busy`; return to `IDLE` when `CTRL.clear_done` is asserted.

TABLE 4.1: Accelerator FSM states and their actions.

on. When the last row finishes, the whole tile is written back and the column offset advances by TN; when the last tile column finishes, the accelerator raises `STATUS.done`. The sub-phases inside `ROW_LOAD` and `NZ_FETCH` exist only to absorb the one-cycle BRAM read latency: the address is driven on one cycle and the data is consumed on the next.

A first-order cycle model per tile column is

$$T_{\text{tile}} \approx M_{\max} + M \cdot C_{\text{row}} + \text{NNZ} \cdot (C_{\text{nz}} + TN) + M \cdot TN \cdot C_{\text{wb}},$$

with $C_{\text{row}} \approx 5$, $C_{\text{nz}} \approx 5$, and $C_{\text{wb}} \approx 1$. The model is deliberately approximate: it ignores writeback overlap and assumes no stalls. Its purpose is to expose the weight of the $\text{NNZ} \cdot TN$ term. At $TN = 8$, filling `B_Seg` costs eight cycles per nonzero, which dominates the other terms in every realistic case. Section 4.6 explains why, and Section 4.8 attacks that term directly.

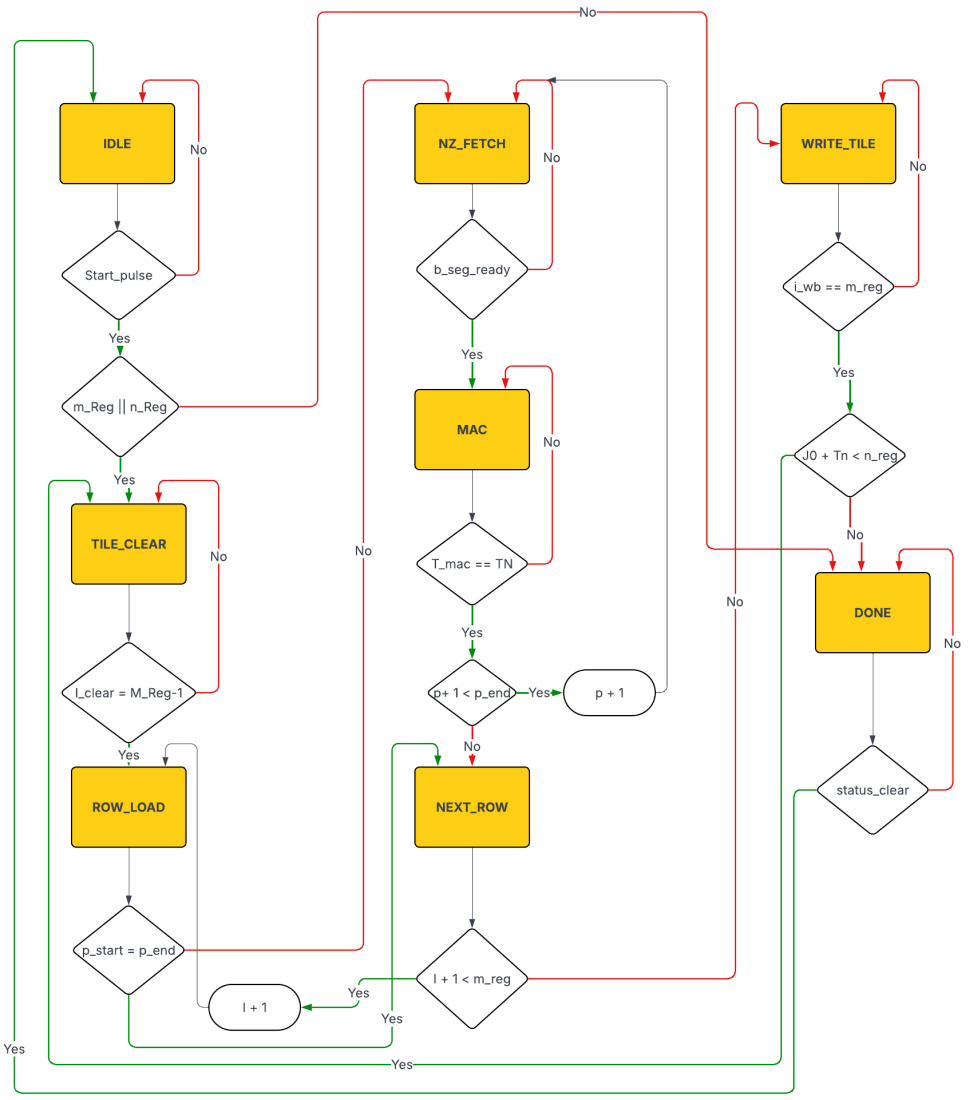

FIGURE 4.3: Simplified accelerator finite state machine. Each major phase corresponds to one row in Table 4.1. Sub-phase transitions within `ROW_LOAD` and `NZ_FETCH` absorb the one-cycle registered RAM output latency.

4.4 The Choice of Tile Width

This section explains why the tile width is set to eight. TN controls four things at once: how many MAC lanes run in parallel, how large the accumulator array is, how many RAM reads are needed to fill `B_Seg`, and how hard the design is to route at 50 MHz. Table 4.2 lists the alternatives.

At $TN = 1$ there is no parallelism at all. At $TN = 4$ the reuse of each fetched B segment is halved. At $TN = 16$ or 32 the C-tile grows past a thousand registers,

TN	MAC lanes	Acc. registers	B-seg reads/NZ	DSP No.	Notes
1	1	64	1	1	Minimal resources; no parallelism; effectively sequential.
4	4	256	4	4	Moderate parallelism; reduced register pressure.
8	8	512	8	8	**Design point.** Good balance; DSP-mappable; 512-reg array feasible.
16	16	1024	16	16	Doubles resources; routing congestion; timing harder to close.
32	32	2048	32	32	Likely infeasible on Cyclone V; 2048-reg array exceeds practical limits.

TABLE 4.2: Resource and performance implications of different tile widths TN. The accelerator implements $TN = 8$.

eight or more extra DSP lanes have to be routed alongside the CPU, BRAM, UART and GPIO already in the fabric, and trial builds stopped closing timing at 50 MHz. $TN = 8$ is the largest width that met timing in those trials. It also fits eight DSP blocks cleanly, keeps `C_Tile` at 512 registers, and covers the smallest benchmark case ($N = 8$) in a single tile column so no case is penalised by tiling overhead.

4.5 Algorithmic Decomposition: Row-Wise CSR Traversal

This section explains why the accelerator walks A row by row rather than using an outer-product or intersection-based approach.

The datapath and FSM both assume a specific traversal order: rows of A visited one at a time, with tiling only across the dense-output dimension. The alternative was an outer-product decomposition (as in OuterSPACE [8]) or intersection-based skipping (as in ExTensor [10]). Both can deliver higher throughput on particular

sparsity patterns, but both need index-merge networks, multi-port accumulation, and in some cases content-addressable memory. None of that fits inside the SoC budget of this project.

Row-wise CSR keeps three things simple. The FSM only has to track one row pointer at a time. The BRAM Port B access pattern stays mostly sequential with no bank conflicts. And `C_Tile` writeback is a straight sweep with no merge logic. These properties are what allowed the design to be verified by hand in simulation before synthesis.

4.6 Memory Optimisation: the B-Segment Bottleneck

This section explains why the baseline accelerator is memory-bound and why the B-segment fill is the dominant cost.

Operating at 50 MHz with $TN = 8$ parallel MAC lanes, the peak arithmetic throughput is

$$T_{\text{peak}} = 8 \text{ MACs/cycle} \times 50 \text{ MHz} = 400 \text{ MMACs/s}.$$

Port B of the BRAM supplies one 32-bit word per cycle, or about 200 MB/s of operand bandwidth. Filling a $TN = 8$ segment from 32-bit operands costs eight reads, so the MAC lanes wait for operands roughly eight times longer than they compute. The design sits well to the left of the roofline knee introduced in Section 2.8, and any optimisation that does not shrink the B-fill term is capped at a small fraction of the ideal speedup.

Breaking the per-nonzero cycle count down makes this explicit. In the baseline, each nonzero takes about ten cycles of useful work: eight to fill `B_Seg`, plus one cycle of registered read latency on the first word, plus one MAC cycle. Of those ten, the eight fill cycles are entirely memory-bound. The next section looks at what can be done on the arithmetic side; Section 4.8 then addresses the memory side directly.

4.7 DSP-Oriented Parallel MAC Enhancement

This section covers the first optimisation stage, which improves how the eight MAC lanes map onto Cyclone V DSP blocks.

Each DSP block on Cyclone V provides an 18×18-bit signed multiplier with a dedicated accumulator register [20]. Quartus will map an RTL multiplier onto a DSP block, but only when the operand widths and the accumulator structure match what the block expects. In the baseline the eight MACs are written as generic 32-bit multiplications, and depending on register fan-out and routing pressure some lanes were inferred in ALM logic rather than in DSP blocks.

The DSP MAC stage rewrites each lane in the registered-feedback form that Quartus recognises:

$$\texttt{C\_Tile}[i][c] \leftarrow \texttt{C\_Tile}[i][c] + a \cdot B[c],$$

with 32-bit signed operands and a registered accumulator. With this form, the MAC lanes are designed to map more predictably onto dedicated DSP multiply-accumulate resources, reducing the amount of arithmetic implemented in ALM logic. However, this stage does not change the number of B-segment reads per nonzero, so it cannot remove the dominant memory-side bottleneck identified in Section 4.6. The larger performance opportunity therefore comes from reducing the B-fill cost, which is the purpose of the INT8 packed mode in Section 4.8.

4.8 INT8 Mode at the Datapath Level

This section describes what changes in the datapath when INT8 mode is enabled. The goal here is implementation detail; the broader argument for why INT8 was the right choice for this workload appears in Chapter 2.

When `CTRL.int8_mode` is set, three specific things change:

1. **The CSR `values` array is packed as four INT8 per 32-bit word.** For nonzero index p the byte lives at `A_VAL_BASE` $+\lfloor p/4 \rfloor \times 4$, and the controller tracks the byte offset $b = p \bmod 4$ with a two-bit counter.
2. **Matrix B is packed the same way.** The eight values needed to fill `B_Seg` now come from two aligned 32-bit reads instead of eight.

3. **Each selected byte is sign-extended to 32 bits** in a single combinatorial stage before it reaches the multiplier:

$$\texttt{B\_Seg}[4k+b] = \text{sign\_ext}_{8\to32}(\texttt{ram\_rdata}[8(b{+}1){-}1 : 8b]), \quad b \in \{0,1,2,3\}. \tag{4.1}$$

Figure 4.4 shows the byte-select and sign-extend path for one 32-bit read.

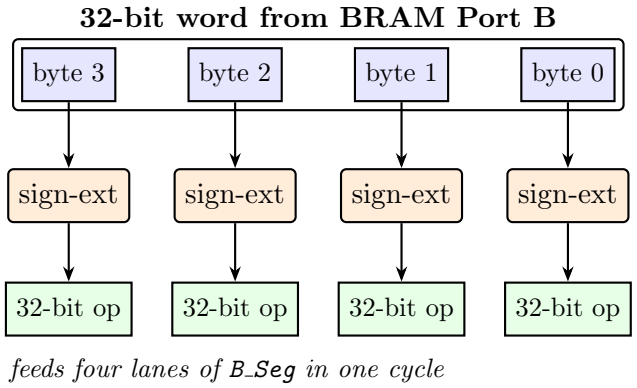

FIGURE 4.4: INT8 byte-select and sign-extension path. One 32-bit read delivers four INT8 operands; each is sign-extended to 32 bits in a single combinatorial stage before it reaches a MAC lane. Two such reads fill all eight lanes of `B_Seg`.

Accumulation and the writeback path are unchanged. Every entry of `C_Tile` stays INT32 throughout the computation, the multiplier still sees a full-precision 32-bit operand (the upper 24 bits are just the sign), and the output matrix C is still written as 32-bit words. The whole change touches only the memory-facing side of the datapath.

4.9 Physical Design Tradeoffs

This section covers the practical FPGA constraints that shaped the design beyond the RTL-level decisions above.

Adding the C-tile register array, the B-segment buffer, and eight DSP-mapped MACs raises the fan-out of several controller signals. The `mac_a` operand in particular has to drive all eight multiplier inputs at once. On Cyclone V, a high-fan-out register combined with the wide BRAM port, the CSR-address paths, and the tile writeback network creates routing congestion that does not appear in RTL simulation.

The target clock is 50 MHz, a comfortable 20 ns period. Cyclone V designs with DSP-mapped multipliers routinely reach 100–200 MHz, so the multiplier is not the critical path. The critical path is the combinatorial logic that generates CSR addresses and drives eight parallel writes into `C_Tile`. After the INT8 packing datapath was introduced, one synthesis run failed to close timing under the default balanced optimisation mode and required the Quartus fitter to be switched to performance-oriented optimisation. That incident is the concrete source of the reflection in Chapter 7 on the hidden costs of parallelism on FPGAs.

Table 4.3 summarises what changed across the three stages. The MAC width, accumulator precision, and output format stay the same (8 lanes, INT32, INT32). The DSP mapping and the per-nonzero B-fill cost are the only things that vary, and those two knobs account for the cycle reduction from baseline to final INT8 build.1

Attribute	Baseline	DSP MAC	INT8 packed
MAC lanes	8	8	8
DSP mapping	Inferred	Explicit	Explicit
B reads per NZ	8	8	2
Operand bit-width	32-bit	32-bit	8-bit (sign-ext to 32)
Accumulator precision	INT32	INT32	INT32
Output format	INT32	INT32	INT32

TABLE 4.3: Architectural attributes across the three design stages. Measured speedups for each stage are reported in Chapter 7.

Chapter 5

Hardware Verification

5.1 Verification Approach

Verification was carried out at two complementary levels: RTL-level simulation before synthesis, and hardware-level functional checking on the DE1-SoC after programming. This two-level approach is standard in digital design practice [21]. RTL simulation is fast and exposes every internal signal, but it cannot by itself guarantee that synthesis, placement, and routing have preserved functional behaviour; hardware-level checking closes that gap by exercising the complete SoC on the physical device. A discrepancy at either level is sufficient to reject a result, and every cycle count reported in Chapter 6 comes from a run that passed both. Figure 5.1 summarises the overall flow.

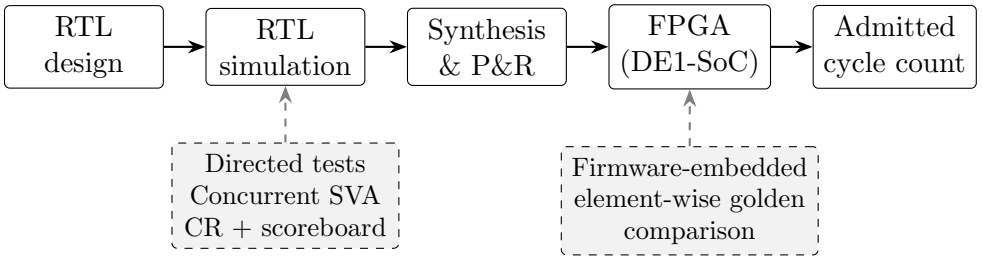

FIGURE 5.1: Two-level verification flow. Each transition between stages is gated by its associated check set; only cycle counts from runs that pass every applicable check are admitted as results.

5.2 RTL Simulation

The simulation tier combines the three verification styles typical of industry-grade digital flows: directed tests with hand-computed golden outputs, concurrent SystemVerilog Assertions (SVA) for always-on protocol and liveness monitoring, and constrained-random stimulus paired with a software scoreboard for exploring the input space beyond what directed tests reach. Four SystemVerilog testbenches share a common stimulus and check framework, three at the unit level and one at the full-SoC integration level. Their scope and the methods applied in each are summarised in Table 5.1.

Testbench	Scope	Methods applied
Datapath unit TB	Arithmetic core in isolation	Directed, CR + scoreboard, SVA
Control-FSM unit TB	FSM with RAM model and datapath stub	Directed, SVA
Accelerator-top TB	MMIO, FSM and datapath together	Directed, CR + scoreboard, SVA, coverage
Integration TB	PicoRV32, accelerator and 64 KB RAM	Firmware-driven, SVA

TABLE 5.1: The four SystemVerilog testbenches that make up the RTL simulation tier, their DUT scope, and the verification methods applied in each.

5.2.1 Unit testbenches

The datapath testbench drives the arithmetic core in isolation, covering B-segment load, MAC accumulation, C-tile clear, signed-operand behaviour, and ReLU on positive, zero, and negative inputs; a constrained-random test then randomises twenty MAC operations per iteration and compares the full tile against a software scoreboard after each batch. The control-FSM testbench instantiates the controller together with a one-cycle-latency RAM model and a datapath stub, and verifies that the FSM enters and exits its busy region within a bounded number of cycles and that its RAM-side signals honour single-port, alignment, and reset-quiescence invariants. The accelerator-top testbench closes the loop between the MMIO front-end, the FSM, and the datapath; a transaction object with random M, K, N, and ReLU enable drives twenty end-to-end runs per seed, each read back and compared to its expected output, with a functional covergroup recording which configuration combinations have been exercised.

5.2.2 Assertion-based checking

Every testbench carries a fixed library of concurrent SVA properties that guard invariants holding for the entire simulation rather than for any one stimulus. These invariants fall into five categories: mutual exclusion of the datapath command signals (the clear, *B*-segment write, MAC, and writeback strobes cannot fire simultaneously); 32-bit address alignment on every RAM access; quiescence of the RAM interface during reset; exclusivity between the `BUSY` and `DONE` status bits; and liveness, which requires every rising edge of `BUSY` to be followed by a falling edge within a bounded number of cycles. The same SVA set is shared across the FSM, accelerator-top, and integration testbenches, so any RTL change that breaks one of these contracts is caught by the entire suite at once.

5.2.3 Constrained-random stimulus and scoreboarding

The constrained-random tests randomise transaction objects whose fields cover the legal configuration space, drive the DUT, and compare the resulting output against a parallel software model updated in lock-step. A functional covergroup samples the exercised configurations after each iteration, so coverage convergence is visible during regression and seed counts can be extended until every bin is hit. The scoreboard itself is an element-wise check over the DUT's full output; any mismatch is reported with the exact index of the offending element and halts the test.

5.2.4 Integration testbench

The integration testbench instantiates PicoRV32, the accelerator, and a 64 KB behavioural RAM, preloads the compiled firmware image, releases reset, and lets the firmware execute its complete benchmark sequence until the CPU writes a sentinel value to a pre-agreed address. This closes the loop between the software control flow of Chapter 6 and the hardware described in Chapter 4: any firmware change that breaks the MMIO contract, any RTL change that violates the firmware's timing assumptions, or any memory-map misconfiguration fails immediately in simulation, before reaching a synthesis run. Additional SVA properties guard the shared memory fabric, most importantly address-decode exclusivity between the RAM and MMIO regions and bounded progress on the CPU memory handshake.

5.3 Hardware-Level Verification

Hardware-level verification is embedded directly inside the firmware benchmark harness described in Section 6.1.1. For each of the 12 cases and at each of the three accelerator stages, firmware generates deterministic inputs from fixed seeds, computes the quantised CPU reference, runs the accelerator on the same inputs, and performs an element-by-element comparison between the accelerator output and the software reference. Because CPU and accelerator apply the same symmetric INT8 quantisation to the same data and accumulate identical INT32 partial sums, their outputs must agree exactly; a single mismatch flags the case as FAIL and no cycle count is recorded.

Taken together with the simulation tier, this yields three independent layers of functional checking before any performance measurement is admitted. Concurrent SVA guards low-level RTL contracts; randomised scoreboards probe the configuration and operand space; and element-wise golden comparison on real silicon certifies that the synthesised, placed, and routed design produces the correct output under real timing. The benchmark inputs, cycle-count definitions, and the resulting measurements are treated as evaluation methodology and are presented in Chapter 6.

Chapter 6

Firmware, Benchmarking and Evaluation

This chapter is the project's experimental half. It describes the bare-metal firmware that drives the accelerator, the benchmark suite that exercises it, and the measured results produced on hardware. The order follows the order of the work: the firmware was built first, the benchmark cases were designed once the firmware was stable, and the cycle-count and correctness numbers reported at the end were produced by running that firmware and that suite on the DE1-SoC.

6.1 Firmware Benchmark Harness

The bare-metal firmware drives the entire benchmark loop: matrix generation, data preparation, accelerator configuration, timing, correctness checking, and result reporting. It runs directly on PicoRV32 from address `0x0000_0000` with no operating system. A short RISC-V assembly bootstrap sets the stack pointer to the top of the 64 KB RAM, clears the BSS section, and enters `main()`, which iterates over the 12 test cases and, for each, drives matrix generation, the software reference, the accelerator launch, correctness comparison, and UART reporting. The driver wraps the MMIO register interface behind configuration, start, and polling functions; libc stubs (`memset`, `memcpy`, `memmove`) cover the minimal C library needs. The linker script maps all code and data into the same 64 KB RAM, with the stack growing downward from the top.

Matrix buffers are allocated statically below the stack, so the CSR arrays, dense matrix B, and the output matrices (`c_accel`, `c_cpu`) occupy contiguous regions. The largest benchmark (T6: $64 \times 64 \times 32$, 75% sparsity, NNZ ≈ 1024) consumes roughly

$$\underbrace{65 \times 4}_{\texttt{rowptr}} + \underbrace{1024 \times 4}_{\texttt{colidx}} + \underbrace{1024 \times 4}_{\texttt{values}} + \underbrace{64 \times 32 \times 4}_{B} + \underbrace{64 \times 32 \times 4}_{C} \approx 29\,\text{KB}$$

of matrix data, well within the 64 KB budget for firmware code and stack.

6.1.1 Runtime data preparation

Dense matrix B is generated by a linear congruential PRNG seeded from the benchmark case's `seed` field, with each entry sampled as a signed integer in $[-127, 127]$; this range is directly usable as INT32 in the full-precision path and as the pre-quantisation domain for the INT8 path. A fixed seed makes B deterministic across runs and firmware revisions, which is what allows cycle-count regression comparison across optimisation stages.

Sparse matrix A is built in three phases. Nonzero positions are placed first according to the selected pattern: a uniform pattern gives each row i a target count $n_i = \lfloor K(1 - s/100) \rfloor$ with column indices sampled without replacement; a row-skewed pattern concentrates roughly 30% of the NNZ budget in a minority of rows, simulating social-graph hub nodes; and a clustered pattern biases nonzeros toward rectangular blocks, mimicking the blocked sparsity seen in finite-element problems. Each nonzero is then assigned a random signed value in $[-127, 127]$. The CSR representation is built last: column indices within each row are sorted ascending (required for determinism and for matching the CPU reference), `rowptr` is a prefix sum of per-row counts, and the sorted indices and values are written to `colidx` and `values`. The sort also makes the CSR form canonical across runs, which simplifies manual inspection of accelerator output.

6.1.2 INT8 quantisation and packing

Before an INT8 run, firmware applies symmetric runtime quantisation separately to the CSR values array and to B. For a vector $\mathbf{v}$ of length L, it computes $v_{\max} = \max_i |v_i|$, sets $s = v_{\max}/127$, and quantises each element as $q_i = \text{clip}(\text{round}(v_i/s), -127, 127)$. When $v_{\max} = 0$, all outputs are zero and $s = 1$. This

matches Equation 2.1 from Section 2.6. The scale factor is discarded immediately: because the correctness check compares accelerator output against the *quantised* CPU reference, s need never be stored, which keeps both the accumulator logic and the verification path scale-free.

After quantisation, four consecutive INT8 values are packed into a single 32-bit word in little-endian byte order:

```
for (i = 0; i < len; i += 4) {
    word = ((uint32_t)(uint8_t)q[i+0])        |
           ((uint32_t)(uint8_t)q[i+1] << 8)   |
           ((uint32_t)(uint8_t)q[i+2] << 16)  |
           ((uint32_t)(uint8_t)q[i+3] << 24);
    dst[i/4] = word;
}
```

B is packed row by row, so a single 32-bit RAM read delivers four aligned INT8 values that populate four lanes of the B-segment buffer after sign extension, as described in Section 4.3.

6.1.3 Software baseline

The software reference is deliberately written in the simplest possible form. The full-precision inner structure is:

```
for (row = 0; row < M; row++) {
    for (p = rowptr[row]; p < rowptr[row+1]; p++) {
        col = colidx[p];
        a   = values[p];
        for (n = 0; n < N; n++)
            C[row*N + n] += a * B[col*N + n];
    }
}
```

Listing 6.1.3: PicoRV32 software SpMM (full precision).

Only the M-extension `MUL` instruction is used; there is no SIMD, no loop unrolling, and no software prefetching. The baseline is deliberately simple: its role is not to

maximise CPU throughput but to provide a credible, readable reference against which the accelerator's structural advantage can be measured. The INT8 reference (`spmm_cpu_q8`) has the same structure but operates on signed 8-bit operands with INT32 accumulation, and is the version used for correctness comparison against the INT8 accelerator.

6.1.4 Accelerator launch and reporting

The driver exposes three entry points: `accel_configure()` writes the dimension and base-address registers, `accel_start_mode()` issues the control write with the `start` pulse and the `relu_en`/`int8_mode` bits, and `accel_wait_done()` polls `STATUS.done`. A launch wrapper chains them into the sequence used on every benchmark case: write `clear_done`, configure, capture t_0, issue `start`, poll, capture t_1, write `clear_done` again, and report $t_1 - t_0$ as the accelerator cycle count.

Results are emitted as CSV over the on-chip UART at 115200 baud (50 MHz / 434 cycles per bit); each line carries the test ID, M, K, N, sparsity percentage and pattern, NNZ count, CPU and accelerator cycle counts, derived speedup, and a PASS/FAIL verdict. A condensed view is also driven onto the DE1-SoC's six seven-segment displays, with the upper pair showing CPU cycles scaled by 64, the middle pair raw accelerator cycles, and the lower pair the integer speedup, so a running benchmark can be read at a glance without a host terminal. The LEDs mirror the busy/done signals, lighting during execution and clearing when computation completes.

Correctness is checked inline after each accelerator run by comparing `c_accel` against the INT8 CPU reference `c_cpu` element by element:

```
for (i = 0; i < M*N; i++) {
    if (c_accel[i] != c_cpu[i]) {
        pass = 0; break;
    }
}
```

A full-precision CPU result is also computed in parallel but untimed, and serves as a sanity reference for inspecting quantisation error. Because CPU and accelerator apply the same symmetric INT8 quantisation to the same data and accumulate identical INT32 partial sums, their outputs must match exactly; any mismatch indicates a hardware bug rather than a numerical artefact.

6.2 Benchmark Suite

The suite sweeps four independent axes so that any observed trend can be interpreted analytically rather than anecdotally: matrix size, which governs amortisation of fixed setup overhead; dense width N, which sets the number of tile columns and the ratio of tile-management to useful work; sparsity, which determines NNZ and therefore per-row arithmetic load; and nonzero distribution pattern, which exposes sensitivity to the spatial layout of NNZ. Table 6.1 lists all 12 cases with the axis each primarily addresses. The same cases are used unchanged across all three stages; this is what makes stage-to-stage comparison meaningful.

Test ID	$M \times K \times N$	Sparsity	Pattern	Purpose
T1	$8 \times 8 \times 8$	75%	Uniform	Small debug/sanity case; establishes minimum speedup in the suite.
T2	$16 \times 16 \times 8$	75%	Uniform	Small size-scaling point; 4× more arithmetic than T1.
T3	$32 \times 32 \times 8$	75%	Uniform	Medium size-scaling point.
T4	$64 \times 64 \times 8$	75%	Uniform	Largest square case; maximum M for the current $M_{\max} = 64$.
T5	$64 \times 64 \times 16$	75%	Uniform	Tests effect of doubling dense width relative to T4.
T6	$64 \times 64 \times 32$	75%	Uniform	Tests effect of quadrupling dense width; 4 tile columns.
T7	$32 \times 32 \times 32$	50%	Uniform	Low sparsity (dense); more nonzeros per row than T8/T9.
T8	$32 \times 32 \times 32$	75%	Uniform	Mid-sparsity reference; matches T10/T11 except pattern.
T9	$32 \times 32 \times 32$	90%	Uniform	Very sparse; exposes overhead-to-arithmetic ratio.
T10	$32 \times 32 \times 32$	75%	Row-skewed NNZ	Same NNZ as T8; row imbalance sensitivity.
T11	$32 \times 32 \times 32$	75%	Clustered NNZ	Same NNZ as T8; block locality sensitivity.
T12	$64 \times 64 \times 32$	90%	Uniform	Large sparse stress case; wide output and few nonzeros.

TABLE 6.1: Complete benchmark input set used across all recorded evaluation stages.

6.3 Cycle-Count Definitions and Measurement Method

CPU cycle counts bracket the timed software SpMM function only: the outer row loop, CSR index reads, inner dense-column MAC loop, and the reads and writes to B and C. Matrix generation, quantisation, and output-buffer clearing

sit outside the bracket and are deliberately excluded. Accelerator cycle counts bracket the complete offload call and therefore include MMIO configuration, the `start` pulse, hardware execution, and the final polling loop. This is an end-to-end offload latency from the CPU's perspective, appropriate for user-visible benefit but explicitly not a pure datapath-throughput figure.

Timing uses the PicoRV32 `rdcycle` CSR, which returns the lower 32 bits of a hardware cycle counter clocked at 50 MHz. The read is wrapped in a short inline-assembly stub:

```
static inline uint32_t read_cycles(void) {
    uint32_t cyc;
    asm volatile("rdcycle %0" : "=r"(cyc));
    return cyc;
}
```

The counter wraps at 2^{32} cycles, roughly 85 seconds at 50 MHz. The largest benchmark completes in under two million cycles, so wrap-around is a non-issue and firmware needs no 32-to-64-bit extension.

Speedup is defined as

$$\text{Speedup} = \frac{T_{\text{CPU}}}{T_{\text{ACCEL}}},$$

with both times in CPU cycles on the same 50 MHz clock. For the RISC-V comparisons, T_{CPU} is the timed PicoRV32 software result (INT32 for the baseline and DSP stages, INT8 for the INT8 stage). For the Cortex-M0 comparison, T_{CPU} is derived from the processor's published CPI model applied to the same benchmark definitions rather than measured on physical hardware; Section 6.8 discusses the implications.

6.4 Baseline Accelerator Results

Table 6.2 gives the full baseline accelerator results against the PicoRV32 software reference. Even unoptimised, the accelerator delivers a consistent speedup, rising from 12.12× at T1 to 18.12× at T4. T1 is the worst case because its 8×8 matrices are too small to amortise the fixed FSM overhead (the $M_{\max} = 64$ C-tile clear, the row loads, and the final writeback), which dominates the cycle count at this size. Section 6.9 analyses the amortisation pattern across T1–T4.

Test ID	$M \times K \times N$ / Sparsity	CPU cycles	Accel cycles	Speedup	Result
T1	$8 \times 8 \times 8$, 75%	10,373	856	12.12×	PASS
T2	$16 \times 16 \times 8$, 75%	36,573	2,318	15.78×	PASS
T3	$32 \times 32 \times 8$, 75%	136,877	7,809	17.53×	PASS
T4	$64 \times 64 \times 8$, 75%	529,101	29,195	18.12×	PASS
T5	$64 \times 64 \times 16$, 75%	986,829	58,112	16.98×	PASS
T6	$64 \times 64 \times 32$, 75%	1,902,285	115,980	16.40×	PASS
T7	$32 \times 32 \times 32$, 50%	951,213	58,112	16.37×	PASS
T8	$32 \times 32 \times 32$, 75%	491,693	30,470	16.14×	PASS
T9	$32 \times 32 \times 32$, 90%	215,263	13,844	15.55×	PASS
T10	$32 \times 32 \times 32$, 75%, row-skewed	491,693	30,470	16.14×	PASS
T11	$32 \times 32 \times 32$, 75%, clustered	491,693	30,470	16.14×	PASS
T12	$64 \times 64 \times 32$, 90%	800,155	49,663	16.11×	PASS

TABLE 6.2: Baseline PicoRV32 software versus baseline accelerator results. Speedup ranges from 12.12× to 18.12× across the 12-case suite.

6.5 Parallel DSP MAC Enhancement Results

Table 6.3 gives the results after the DSP-oriented MAC restructuring described in Chapter 4. Total accelerator cycles fall from 427,299 to 324,568, a 1.32× improvement, with every case improving over its baseline counterpart. The gain is consistent but modest, and that is itself informative: even with optimally scheduled DSP arithmetic, the accelerator is not compute-bound. The B-segment memory transaction overhead ($TN = 8$ reads per nonzero at 32-bit precision) remains the dominant cost, and no restructuring of the arithmetic alone can remove it.

Test ID	CPU cycles	Accel cycles	Speedup	Result
T1	10,373	737	14.07×	PASS
T2	36,573	1,859	19.67×	PASS
T3	136,877	6,024	22.72×	PASS
T4	529,101	22,021	24.03×	PASS
T5	986,829	43,781	22.54×	PASS
T6	1,902,285	87,301	21.79×	PASS
T7	951,213	43,781	21.73×	PASS
T8	491,693	23,296	21.11×	PASS
T9	215,263	10,988	19.59×	PASS
T10	491,693	23,296	21.11×	PASS
T11	491,693	23,296	21.11×	PASS
T12	800,155	38,188	20.95×	PASS

TABLE 6.3: Results after enabling the parallel DSP-oriented MAC enhancement. Total accelerator cycles fell from 427,299 to 324,568, a 1.32× reduction.

6.6 INT8 Packing Results

Table 6.4 gives the final INT8 results, the largest performance step in the project by a wide margin. Total accelerator cycles fall from 324,568 to 148,737, a 2.18× reduction. The mechanism is exactly what the roofline analysis of Chapter 2 predicted: cutting B-segment reads per nonzero from 8 to 2 reduces the dominant memory-bandwidth cost by a factor of four on the reads themselves, and the overall cycle-count impact is 2.18× because not every cycle is memory-bound. Row loads, nonzero index fetches, and the final C-tile writeback are fixed overheads that do not shrink with INT8.

Test ID	CPU cycles (INT8)	Accel cycles	Speedup	Result
T1	10,735	567	18.93×	PASS
T2	38,087	1,111	34.28×	PASS
T3	142,999	2,964	48.25×	PASS
T4	553,655	9,747	**56.80**×	PASS
T5	1,044,151	19,216	54.34×	PASS
T6	2,025,143	38,171	53.05×	PASS
T7	1,012,631	19,216	52.70×	PASS
T8	522,391	11,039	47.32×	PASS
T9	227,481	6,109	37.24×	PASS
T10	522,391	11,039	47.32×	PASS
T11	522,391	11,039	47.32×	PASS
T12	849,333	18,519	45.86×	PASS

TABLE 6.4: Final INT8-packed implementation results. Peak speedup of 56.80× on T4; average speedup of 45.28× across the 12-case suite. Total accelerator cycles fell from 324,568 to 148,737, a 2.18× reduction over the DSP stage.

6.7 Stage-Level Summary

Table 6.5 consolidates the per-stage results into a single view. All 12 benchmark cases pass their element-by-element correctness checks at every stage, on hardware. Two observations stand out. First, each stage improved on the previous one without a correctness regression; this is a non-trivial outcome in a hardware project where incremental FSM and datapath changes can readily introduce off-by-one or sign-extension bugs. Second, the largest gain came from reducing memory traffic (INT8 packing: 2.18×) rather than from increasing arithmetic parallelism (DSP MAC: 1.32×), directly confirming the memory-bandwidth bottleneck hypothesis developed in Chapters 2 and 4.

Stage	Avg speedup	Best case	Total accel cycles	Reduction vs prev.
Baseline accelerator	16.12×	T4: 18.12×	427,299	n/a
Parallel DSP MAC	20.87×	T4: 24.03×	324,568	1.32×
INT8 packed	45.28×	T4: 56.80×	148,737	2.18×

TABLE 6.5: Summary of the three staged RISC-V accelerator evaluations across the full 12-case benchmark suite.

6.8 Comparison Against ARM Cortex-M0

Table 6.6 compares the INT8 accelerator against a software SpMM reference whose cycle counts come from an instruction-level model of the ARM Cortex-M0 rather than a measurement on physical hardware. The model applies the processor's published CPI characteristics (predominantly single-cycle arithmetic, 2–3 cycle word loads) to the same benchmark definitions used elsewhere. The Cortex-M0 is included because it is a substantially more efficient in-order baseline than PicoRV32 and is closer to the class of embedded core with which an FPGA-attached SpMM block would plausibly be paired. Speedups are correspondingly lower; the accelerator still reaches 11.92× on T4, a meaningful result for a small FPGA-attached block. Being model-derived, these figures should be read as an order-of-magnitude cross-check rather than a same-platform measurement, and the PicoRV32 results remain the primary empirical evidence of this chapter.

Test ID	Cortex-M0 cycles	Accel cycles	Speedup	Result
T1	2,952	567	5.21×	PASS
T2	9,448	1,111	8.50×	PASS
T3	31,912	2,964	10.77×	PASS
T4	116,200	9,747	**11.92**×	PASS
T5	207,592	19,216	10.80×	PASS
T6	390,376	38,171	10.23×	PASS
T7	195,240	19,216	10.16×	PASS
T8	104,488	11,039	9.47×	PASS
T9	46,892	6,109	7.68×	PASS
T10	99,748	11,039	9.04×	PASS
T11	102,209	11,039	9.26×	PASS
T12	169,710	18,519	9.16×	PASS

TABLE 6.6: Model-based ARM Cortex-M0 software SpMM reference versus the final INT8 accelerator. The Cortex-M0 is a stronger baseline than PicoRV32, so speedups are correspondingly lower; the accelerator still achieves up to 11.92× speedup.

6.9 Result Analysis

6.9.1 Scaling with problem size (T1–T4)

Figure 6.1 [placeholder] plots speedup against matrix size for the three stages. Speedup rises consistently from T1 to T4 in all three, and in the INT8 stage grows from 18.93× at T1 ($8 \times 8 \times 8$) to 56.80× at T4 ($64 \times 64 \times 8$). The increase is sublinear in problem size: an eight-fold increase in arithmetic from T1 to T2 yields only 1.81× more speedup, and the four-fold increase from T3 to T4 yields only 1.18× more, indicating that the accelerator is approaching the plateau where fixed overhead is a small fraction of the total. The driver is the ratio of useful MAC work to FSM overhead: T1 has roughly 12 nonzeros at 75% sparsity, and the C-tile clear alone (64 cycles for $M_{\max} = 64$) dominates, whereas T4's 1024 nonzeros spread the same fixed setup cost over far more useful work.

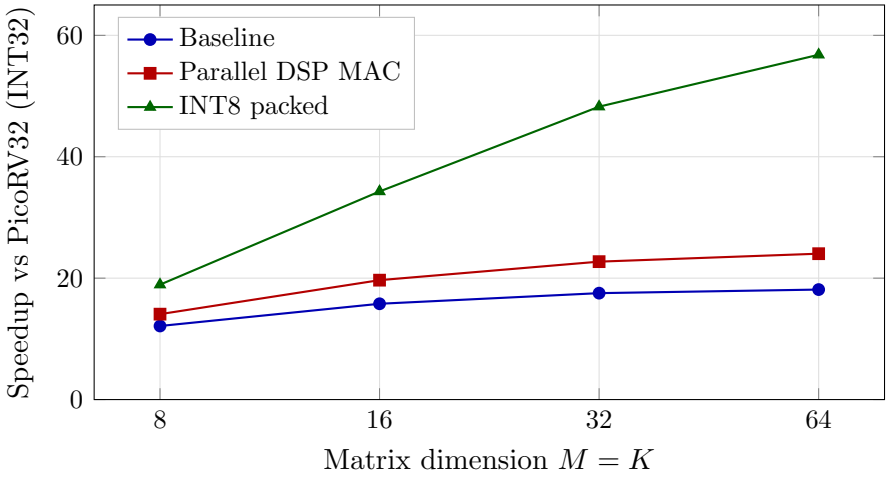

FIGURE 6.1: Speedup versus matrix size for tests T1–T4 (fixed $N = 8$, 75% uniform sparsity) across the three optimisation stages. All three curves rise as problem size grows, confirming that fixed setup overhead (tile clear, row-bound load, final writeback) is amortised effectively at larger sizes. The baseline and DSP curves flatten by T4, whereas the INT8 curve is still climbing steeply, indicating that INT8 packing not only shifts the absolute speedup upward but also extends the useful scaling regime.

6.9.2 Effect of dense width (T4–T6)

The sequence T4 ($N = 8$), T5 ($N = 16$), T6 ($N = 32$) tests how speedup changes as more tile columns are processed at fixed $M = K$. In the INT8 stage speedup moves from 56.80× at T4 through 54.34× at T5 to 53.05× at T6, falling only

modestly as N grows. The accelerator handles multiple tile columns by repeating the row traversal per column, separated by tile-clear and writeback phases, and the CPU's inner loop also scales linearly with N; the near-constant speedup therefore reflects linear scaling on both sides and a preserved structural advantage. The mild decline reflects the relative cost of the tile-management phases: at four tile columns, four separate clear/writeback pairs run sequentially, and this fixed per-column overhead accumulates on the accelerator's side but not the CPU's.

6.9.3 Effect of sparsity (T7–T9)

The sequence T7 (50%), T8 (75%), T9 (90%) fixes $M = K = N = 32$ and sweeps sparsity, giving $52.70\times \rightarrow 47.32\times \rightarrow 37.24\times$ in the INT8 stage. Speedup falls as the matrix becomes sparser because the accelerator's per-row overhead (tile clear, row-bound load) is paid regardless of row population, whereas the CPU simply skips empty rows. At 90% sparsity most rows of A are empty yet the accelerator still visits each of the 32, contributing fixed traversal cycles. Even so, $37.24\times$ at T9 is a substantial speedup and shows the design remains practical well into the sparsity regimes typical of GNN workloads. Real social-graph GNN matrices often exceed 99% sparsity, where absolute cycle counts are low and the relative speedup may fall further, but the acceleration in absolute time would still be valuable. Figure 6.2 plots the three-stage trend.

6.9.4 Pattern sensitivity (T8, T10, T11)

T8, T10, and T11 share dimensions, sparsity, and NNZ, differing only in how nonzeros are distributed (uniform, row-skewed, clustered). In the INT8 stage all three report identical accelerator cycle counts of 11,039 and CPU counts of 522,391. This insensitivity is not accidental: the accelerator processes each row sequentially regardless of its nonzero count, and total NNZ, which sets the number of `NZ_FETCH` and `MAC` cycles, is held fixed across the three patterns. Cycle count is therefore dominated by total NNZ rather than by its spatial distribution. At larger scales, load imbalance could become visible, since very dense rows would force many sequential B-segment loads before the next row, but this effect is not triggered at the present benchmark sizes. The Cortex-M0 model shows small but nonzero differences (104,488, 99,748, and 102,209 cycles) consistent with pattern-dependent variation in CSR-index reads and inner-loop iterations; a deeper analysis of the cause is left for future work.

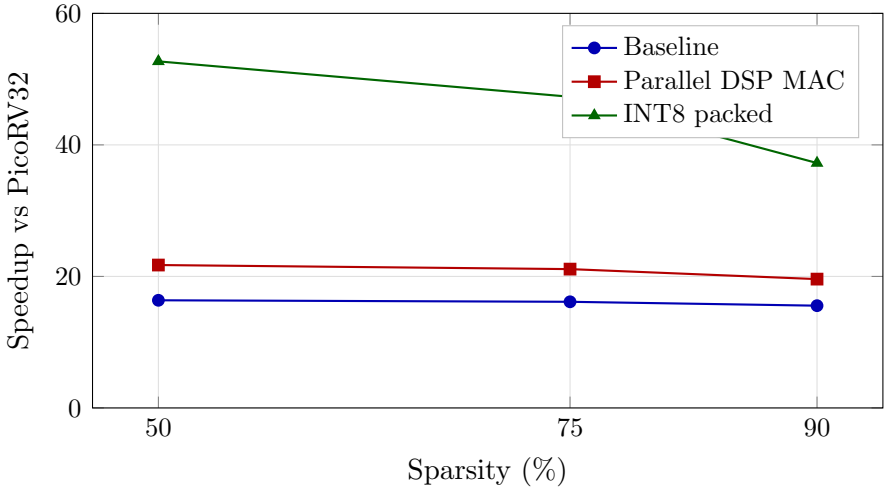

FIGURE 6.2: Speedup versus sparsity for tests T7–T9 (fixed $M = K = N = 32$, uniform pattern) across the three optimisation stages. The baseline and DSP stages are almost insensitive to sparsity because their cycle counts are dominated by per-row overhead that scales with M, not with NNZ. The INT8 stage shows a pronounced decline, as reducing memory-bandwidth cost makes the residual fixed-per-row traversal overhead a larger relative share of the accelerator's total cycles.

6.10 Resource and Timing Evaluation

Table 6.7 gives estimated resource utilisation for the three build variants. Final Quartus post-fit reports were still being produced at the time of writing; the figures below are pre-place-and-route estimates from the Quartus Fitter combined with known design parameters, and are expected to be replaced by exact post-fit numbers in the final revision. Utilisation is modest: roughly 16% of ALMs, 5% of M10K BRAMs, and 9% of DSP blocks. The M10K BRAM count is fixed at 16 across all three stages because it is determined entirely by the shared 64 KB RAM, not by the accelerator's internal buffers, which sit in fabric registers; the DSP count of 8 matches the 8 parallel MAC lanes of the $TN = 8$ datapath. All three builds meet the 50 MHz timing constraint reported by the Quartus timing analyser. The observed headroom on the Cyclone V is a factual property of the current build; concrete plans for using it (for example, a wider tile) are deferred to Chapter 8.

Build	ALMs	Registers	M10K BRAMs	DSP blocks	Fmax (MHz)
Baseline accelerator	≈4,800	≈7,500	16	8	50+
Parallel DSP MAC	≈5,000	≈7,800	16	8	50+
INT8 packed (current)	≈5,200	≈8,000	16	8	50+
Available (Cyclone V)	32,070	128,280	397	87	n/a

TABLE 6.7: Estimated resource utilisation for the three build variants on the Cyclone V 5CSEMA5F31C6. All three designs meet the 50 MHz timing constraint. Final post-fit numbers to be updated in the next revision.

6.11 Fairness, Limitations, and Threats to Validity

The comparison is fair in several respects. CPU and accelerator operate on identical inputs from the same fixed seeds, share the same 64 KB on-chip RAM at the same 50 MHz clock, and are checked against the software reference before any cycle count is admitted. The PicoRV32 baseline uses the M-extension `MUL` for its inner multiplies, so it is not artificially slowed by emulation, and the Cortex-M0 figures come from an instruction-level model applied to the same benchmark definitions rather than a same-platform measurement.

Several limitations bound the claims that can be drawn. The PicoRV32 baseline is a simple in-order core with no cache, no speculation, and no SIMD; against a modern superscalar CPU with vector extensions the measured speedup would be considerably smaller. A more aggressively optimised software implementation (software-pipelined inner loops or explicit B-row reuse, for instance) would also narrow the gap. The baseline's goal is credibility and simplicity, not maximum CPU throughput.

As noted in Section 6.8, the Cortex-M0 comparison is model-based. The model captures the coarse CPI behaviour of a single-issue in-order Cortex-M0 reasonably well for the straight-line arithmetic that dominates CSR SpMM, but it does not capture bus contention, memory-subsystem effects, or compiler-specific code generation on a real part. The Cortex-M0 figures are therefore an order-of-magnitude cross-check, not a like-for-like measurement.

Each reported cycle count is a single recorded value rather than a statistical distribution; counts for deterministic hardware are repeatable, but a min/median/max summary would add confidence. The test matrices are PRNG-generated rather than drawn from real GNN or recommender-system datasets, so although they

cover realistic sparsity levels and pattern structures, behaviour on real workloads may differ. All data lives in the 64 KB on-chip BRAM; real SpMM workloads typically span gigabytes and incur DRAM latency, which would affect CPU and accelerator in different proportions, so the present results reflect the best case of tightly coupled on-chip memory. Accelerator cycle counts include MMIO configuration and polling overhead, which is non-trivial on the smallest cases. Finally, the Quartus post-fit resource and timing reports were still being finalised at the time of writing; the design has been verified to close timing at 50 MHz, but exact ALM, BRAM, and DSP counts across all three variants remain pending. These limitations bound the quantitative claims of the chapter.

6.12 Critical Evaluation

The three-stage experiment gives a clear, quantitative demonstration that the dominant bottleneck in CSR SpMM on this platform is memory bandwidth, not arithmetic throughput. The 1.32× gain from the DSP MAC (an arithmetic improvement), set against the 2.18× gain from INT8 packing (a memory-traffic improvement), is direct experimental evidence of this. The result is consistent with the analysis in Section 2.2: the operational intensity of CSR SpMM with $TN = 8$ 32-bit operands is low, placing the design below the roofline's compute–bandwidth crossover, and INT8 packing shifts intensity upward by reducing operand fetch traffic, moving the design toward the compute-bound region.

Direct comparison of the 56.80× peak against OuterSPACE [8], SpArch [9], or SIGMA [11] would not be meaningful: those systems target orders-of-magnitude larger SpMM on dedicated memory hierarchies. The relevant question is whether a tightly integrated, MMIO-driven FPGA accelerator can deliver useful speedup on a constrained embedded platform; the measured 56.80× peak and 45.28× average over PicoRV32, and up to 11.92× over the Cortex-M0 model, answer in the affirmative. For wider context, Bell and Garland [7] report 2–10× GPU SpMV speedups over scalar CPU code depending on matrix structure; the present accelerator records higher speedups on a simpler SoC because its software baseline is a slow in-order core and because INT8 packing sharply reduces the memory bottleneck.

Read alongside Section 6.11, the results are internally consistent and reproducible; correctness verification at every step confirms that the recorded speedups reflect real computation rather than artefacts of incorrect output; and the stage-to-stage

trend is explained by the same bandwidth argument that motivated the design. The quantitative headline is simple: on this platform and against these baselines, reducing operand traffic through INT8 packing was by a wide margin the highest-leverage optimisation available.

Chapter 7

Reflection

7.1 What Worked Well

Structuring development as a sequence of discrete, measurable optimisation stages (baseline, DSP MAC, and INT8 packing) was the single most valuable decision in the project. A working baseline was established and verified on hardware before any optimisation was attempted, and each subsequent stage changed exactly one aspect of the design: arithmetic parallelism first, then memory traffic. This made it possible to attribute each measured improvement to a specific architectural change, to detect regressions immediately, and to build the evidence-based narrative used throughout Chapter 6.

The deterministic benchmark harness was equally important. Generating all test inputs from fixed seeds at runtime, and computing golden outputs on the same PicoRV32 host, removed the need for off-chip reference data and ensured that the same matrices were used across all three hardware stages. Any functional discrepancy introduced by a hardware or software change would have been caught by the element-by-element comparison on the next rebuild.

The roofline analysis in Chapter 2 correctly predicted before implementation that memory traffic, not arithmetic, would be the dominant bottleneck. The measured 1.32× gain from DSP-based MAC compared to the 2.18× gain from INT8 packing confirmed this diagnosis quantitatively. A design effort that had focused purely on widening the MAC array would have yielded diminishing returns; addressing the memory bottleneck directly produced the largest single performance step.

7.2 What Went Wrong

The most time-consuming debugging episode involved a stale `firmware.mif` file. The FPGA synthesis flow uses this memory initialisation file to preload on-chip RAM with the compiled firmware image. When the file was not regenerated after a firmware change, the board continued to run the previous image while the expected code had in fact been rebuilt. Diagnosing this required noticing that on-board UART output did not match the current source, which was non-trivial when the visible output is only a few bytes. The lesson is that FPGA flows combining independently compiled software and hardware artefacts require explicit dependency management in the build system; a Makefile rule that re-triggers synthesis on `firmware.mif` changes would have prevented the issue entirely.

A second lesson came from increasing routing pressure as parallel logic was added. Quartus reported higher congestion around the C-tile register array and the wide MAC buses, and timing closure failed after the INT8 packing datapath was introduced until the fitter was switched from the default balanced optimisation mode to performance-oriented optimisation. Parallelism in FPGA designs carries costs beyond logic area: routing, placement, and timing closure are interdependent, and a design that fits comfortably in ALMs can still fail to close timing if the wiring density in a local region exceeds the device's capacity. This tension does not appear in RTL simulation and cannot be resolved by functional correctness alone.

7.3 Project Management

The project ran across two semesters. Semester 1 covered the literature review, the design and planning phases, RTL development and integration, and the initial FPGA deployment. Semester 2 covered the two remaining optimisation stages (DSP-oriented MAC and INT8 quantisation), the hardware benchmarking campaign, and report writing. The Gantt charts in Appendix **??** record both the original plan and the way the work actually unfolded.

Semester 1 schedule pressure arose almost entirely from the implementation and integration phases, which ultimately ran roughly two weeks longer than the planned allocation, as noted in Appendix **??**. Three factors contributed. First, RTL verification exposed a combination of syntax errors and functional defects in the early testbenches, each of which required targeted debugging before the unit suites

would pass. Second, the SoC was originally specified with an interrupt-driven completion path from the accelerator to the CPU, but the unforeseen complexity of integrating interrupt handling with the PicoRV32 exception model led to the mechanism being descoped in favour of a simple polling loop on the status register. Third, the plan allocated only a four-day slack window (22–25 December) for unforeseen errors, which was consumed almost immediately once the integration problems began to appear and proved far too short for a project of this scale.

Semester 2 pressure originated mainly from the benchmarking phase. The UART peripheral bundled with the PicoRV32 reference design did not support the cycle-accurate timing readouts required by the measurement methodology in Chapter 5, which blocked progress on the benchmark harness until a replacement was identified. Adopting the open-source `simpleuart` core resolved this and removed the largest single source of benchmarking delay. The stale firmware image bug discussed in the preceding section compounded the pressure by masquerading as functional failure for several days before the root cause was understood. Finally, after the INT8 packing datapath was added, Quartus routing failed to close timing under the default balanced optimisation mode; switching the fitter to performance-oriented optimisation closed timing but added roughly twenty minutes to every full synthesis run, which measurably lengthened the remaining debug iterations. Preserving a fully working, deployable design at the end of every stage, rather than accumulating partially integrated features, was the practice that kept the overall schedule recoverable despite these setbacks.

The overarching lesson is that hardware project timelines are dominated by integration and debug rather than initial design. Treating build-flow discipline (artefact management, synthesis scripts, timing closure) as first-class engineering work from the outset, rather than as polish at the end, would have eliminated most of the schedule pressure experienced in the second half of the project.

Chapter 8

Conclusion

8.1 Summary of Achievements

This project set out to design, implement, and evaluate a RISC-V-attached hardware accelerator for CSR sparse matrix–dense matrix multiplication on the Terasic DE1-SoC, and to demonstrate a measurable speedup over software while remaining reproducible at the register-transfer level. That aim was met. A complete PicoRV32 SoC with a working SpMM accelerator, 64 KB shared dual-port BRAM, MMIO interface, and UART reporting was synthesised and deployed; three recorded optimisation stages (baseline, DSP-enhanced, and INT8 packed) were each verified for correctness on hardware; and a 12-case benchmark harness with deterministic matrix generation, cycle-accurate timing, and element-by-element correctness checking was developed to evaluate them.

The final INT8-packed implementation achieves a peak speedup of **56.80**× over the quantised PicoRV32 software reference and **11.92**× over an ARM Cortex-M0 baseline running the same algorithm, with an average of 45.28× and 9.79× respectively across the 12-case suite. All 12 cases passed correctness checks at every stage.

8.2 Significance

The significance of this work lies not in the absolute numbers compared to large research accelerators, but in what the staged measurement reveals about the

workload. The 1.32× gain from DSP-based MAC versus the 2.18× gain from INT8 packing experimentally confirms that the dominant bottleneck in CSR SpMM on a memory-bandwidth-constrained embedded FPGA is operand fetch traffic, not arithmetic execution. This is a useful result beyond the specific artefact: it indicates that similar embedded accelerators should prioritise format-level and precision-level optimisations over wider MAC arrays. The project also demonstrates that a fully integrated, firmware- driven hardware benchmark harness is a practical alternative to external simulation infrastructure for small-scale accelerator evaluation, and that a useful SpMM accelerator fits comfortably within the resource budget of a Cyclone V device.

8.3 Future Work

The most impactful extensions to the current design would be:

1. **Wider tile width.** INT8 packing reduces the per-nonzero fetch cost enough that widening from $TN = 8$ to $TN = 16$ or $TN = 32$ is now attractive. Doubling the tile width costs an additional two 32-bit reads per nonzero but delivers 2× the MAC throughput per nonzero, and should push the speedup ceiling further.

2. **Ping-pong B-segment prefetch.** Double-buffering the B-segment allows the next segment to be fetched while the current MAC operation completes, overlapping memory latency with arithmetic and removing it from the critical path.

3. **External memory interface.** Extending the design to address off-chip SDRAM or HPS DRAM through the DE1-SoC's HPS bridge would permit realistically sized GNN and recommender-system matrices to be benchmarked, at which point off-chip bandwidth becomes the dominant term and different optimisations become relevant.

4. **Interrupt-driven completion.** Replacing the polling loop with a RISC-V interrupt handler would free the CPU during accelerator execution, enabling pipelined benchmark preparation or post-processing of the previous result.

5. **Power and energy characterisation.** Instrumenting the DE1-SoC rails or using Quartus Power Analyser with simulation-derived toggle files would

add energy efficiency to the evaluation, which is the metric that matters most in the embedded and edge contexts for which this class of accelerator is targeted.

8.4 Closing Remarks

This project has delivered a complete, measured, and reproducible RISC-V FPGA-attached SpMM accelerator that achieves a peak speedup of 56.80× on hardware through three progressively optimised designs. The work confirms that even without exotic hardware resources or advanced scheduling, careful alignment of the memory format, datapath structure, and quantisation strategy with the actual bottleneck of the workload produces large, predictable, and experimentally confirmed performance gains.